

머신러닝 II

01 회귀

목차 01 회귀

1. 회귀 개념
2. 단순 선형 회귀
3. 다중 선형회귀와 다항 회귀
4. 과적합과 정규화
5. 정규화를 적용한 회귀
6. 회귀 알고리즘 평가 지표

1. 회귀 개념



머신러닝

지도학습

비지도학습

강화학습

회귀분석
(Regression)

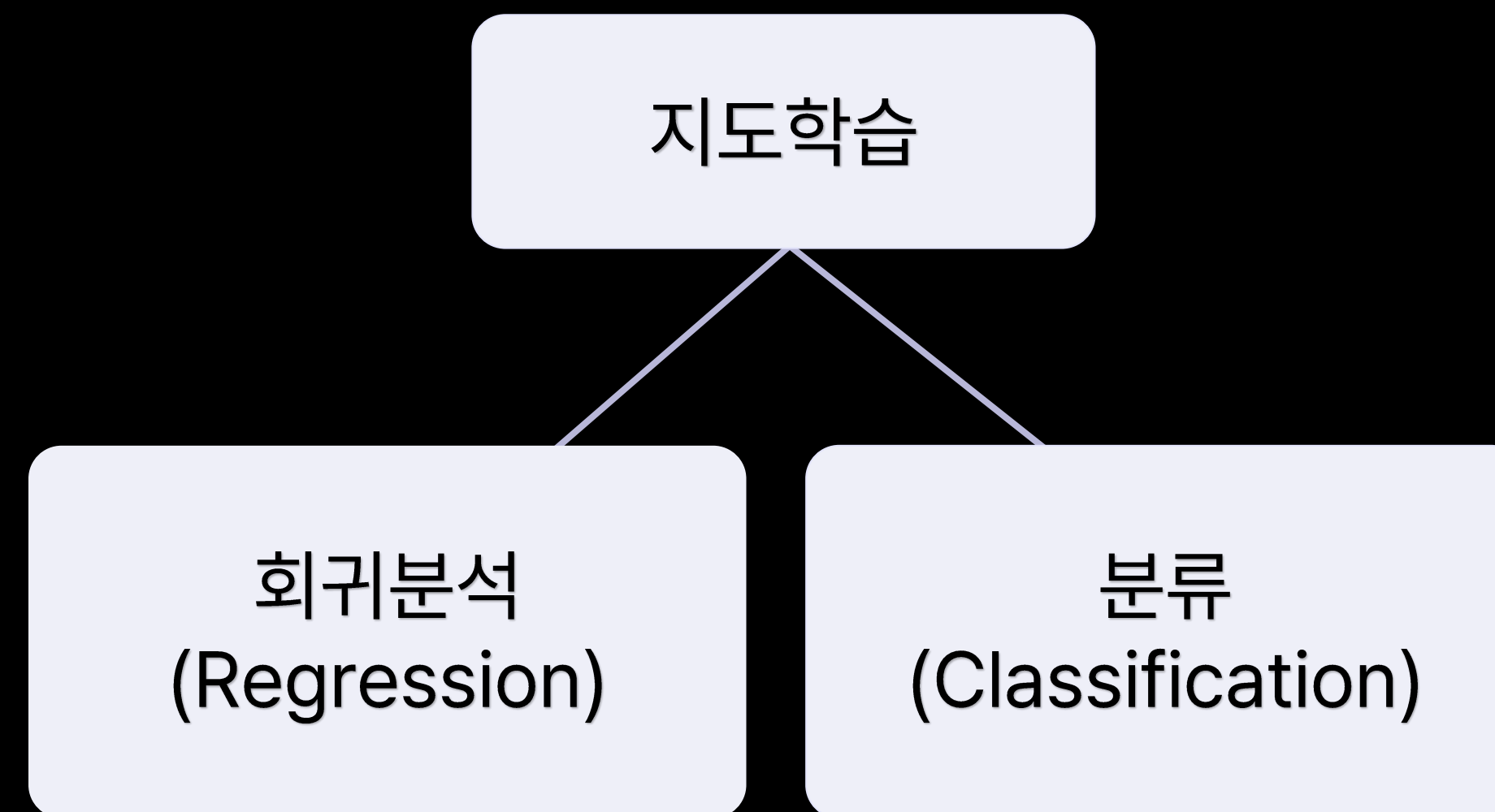
분류
(Classification)

클러스터링
(Clustering)

차원 축소
(Dimensionality
Reduction)

컴퓨터에 **정답이 있는** 데이터를 학습시키는 방법

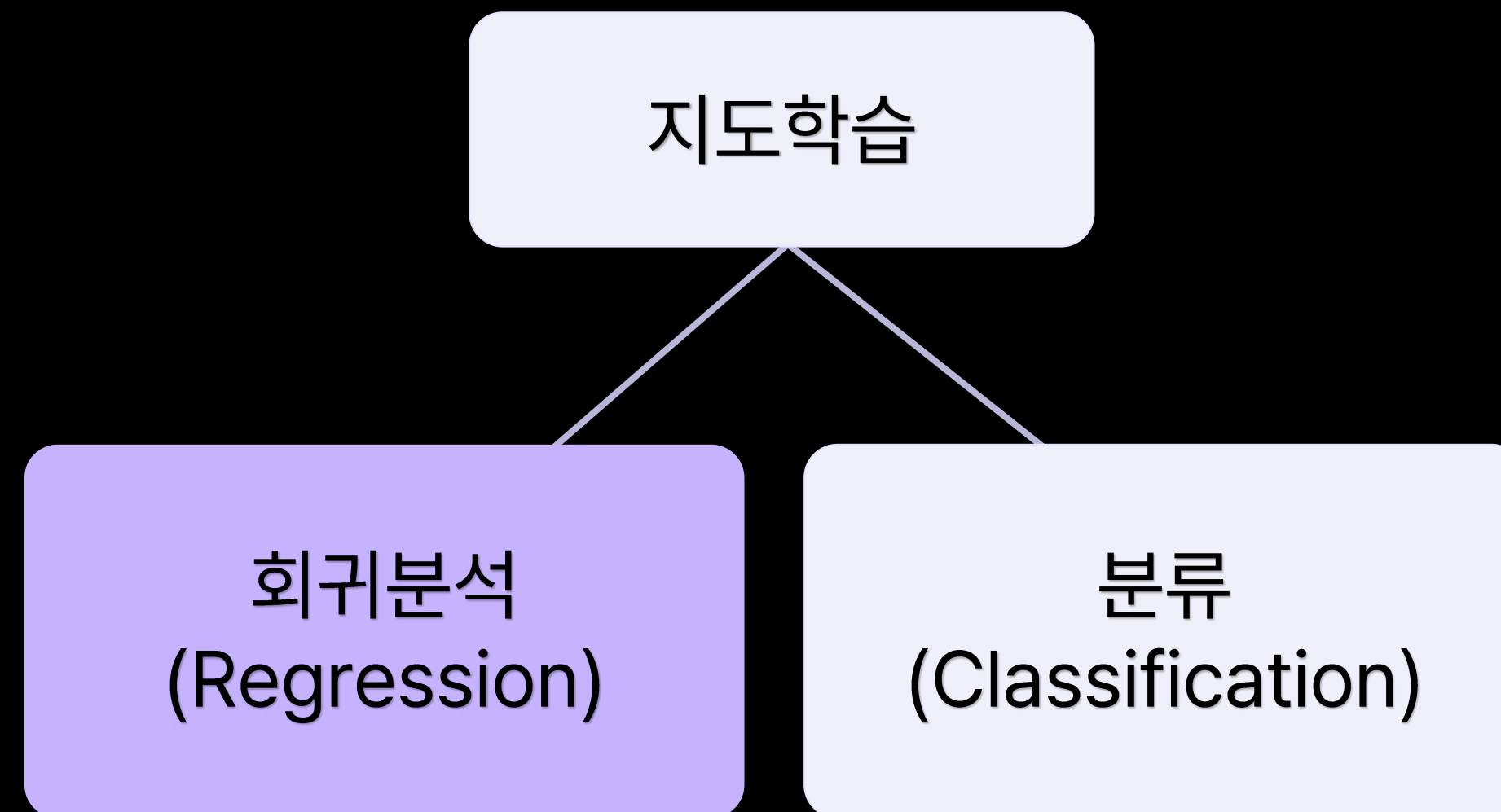
- 주어진 입력으로부터 출력 값을 예측하고자 할 때 사용





주어진 입력 변수 (x) 에 대해 **연속적인 출력 변수 (y)** 를 **예측**하는 알고리즘

(데이터를 가장 잘 설명하는 선을 찾아 입력값에 따른 **미래 결과값을 예측**하는 알고리즘)





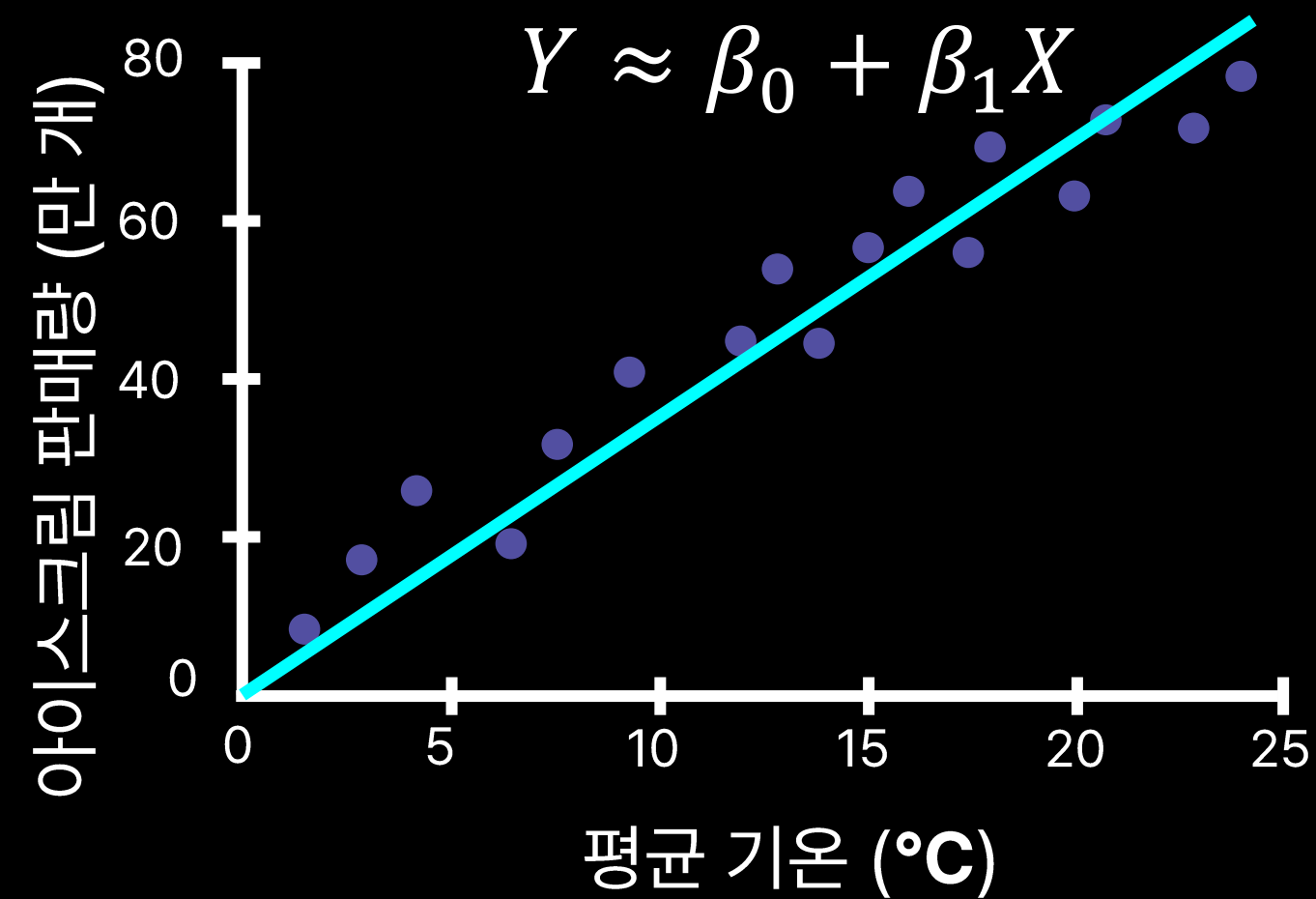
- 아이스크림 가게를 운영하는 주인이라고 가정해보자
- 판매용 아이스크림 주문 시, 예상되는 실제 판매량 만큼만 주문을 원한다.
- 이 때, 만약 **평균 기온 (x)** 을 활용해 **미래 판매량 (y)** 을 예측할 수 있다면?





- 데이터 : X : 과거 평균기온과 그에 따른 Y : 아이스크림 판매량
- 가정 : 평균 기온과 판매량은 **선형적인 관계**를 가지고 있음
- 목표 : 평균 기온에 따른 아이스크림 판매량 예측하기
- 해결 방안 : **회귀 분석 알고리즘**

X	Y
평균 기온(°C)	아이스크림 판매량(만개)
10	40
13	52.3
20	60.5
25	80

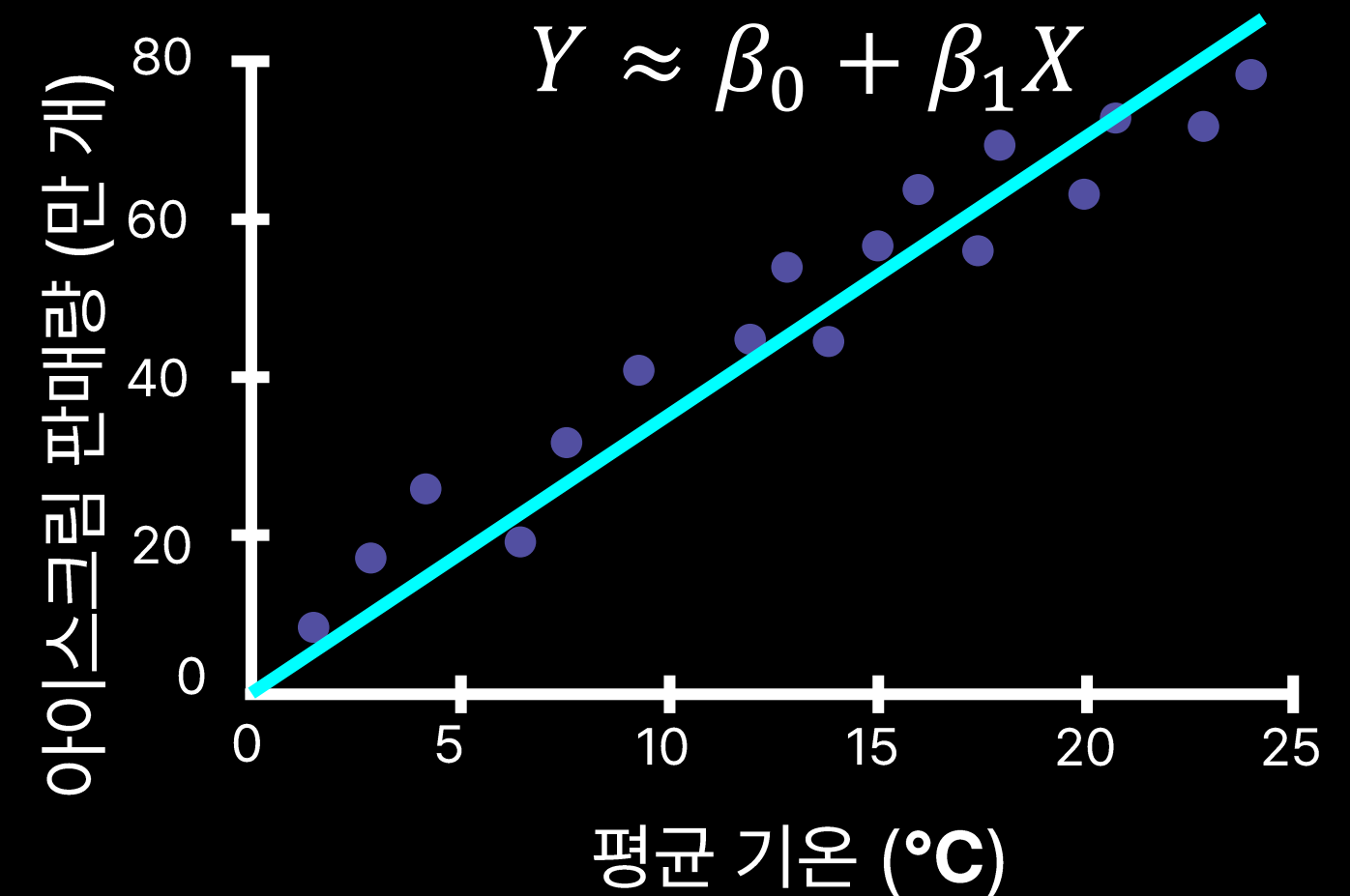


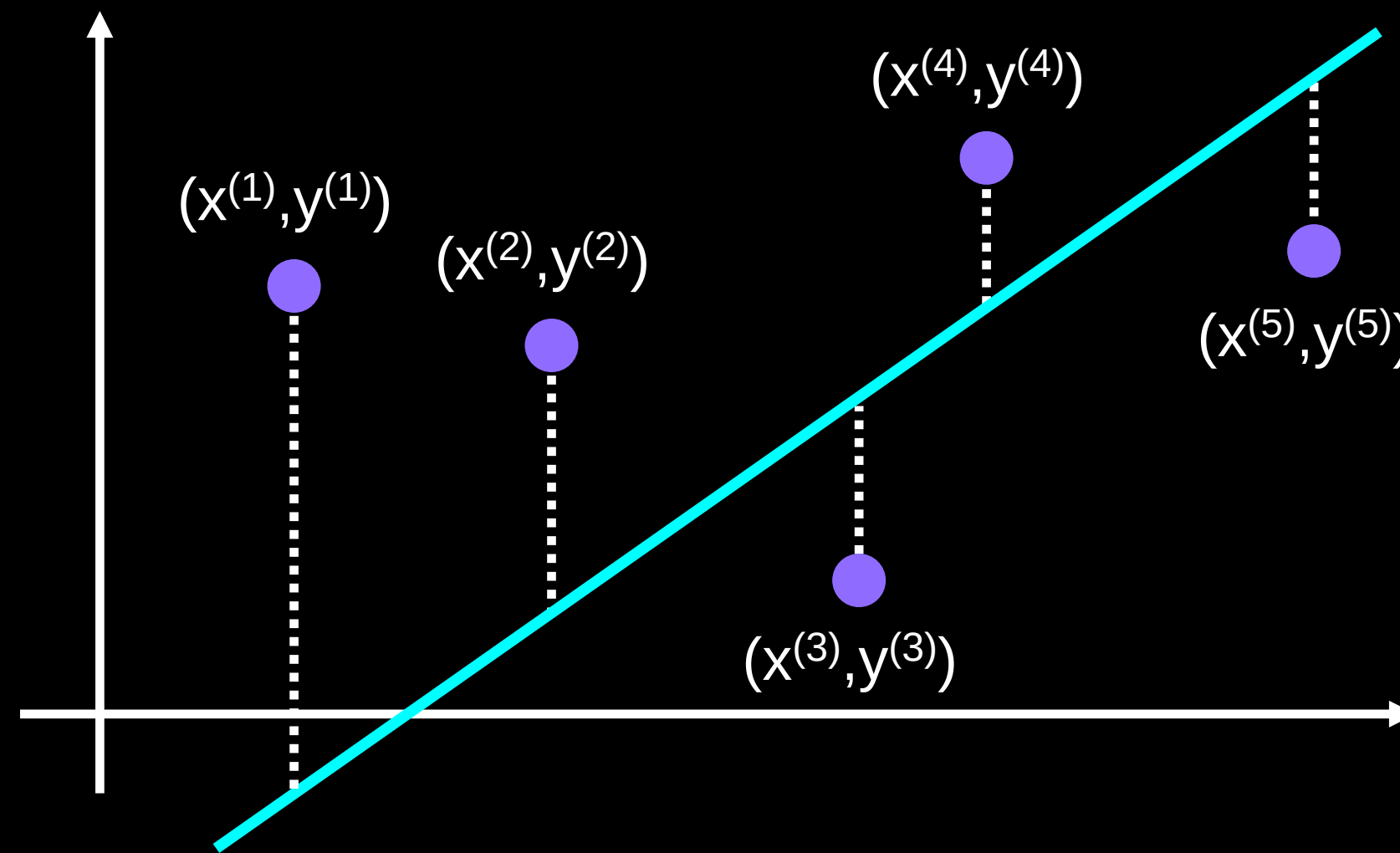
- $Y \approx \beta_0 + \beta_1 X$
- $X = \text{평균 기온}$, $Y = \text{아이스크림 판매량}$
- 적절한 β_0 (y절편)과 β_1 (기울기) 찾기
잘 설명하는 선



데이터를 가장 잘 설명하는 선을 찾아 입력값에 따른 미래 결과값을 예측하는 알고리즘

- 완벽한 예측은 불가능
- 실제 값과 예측 값의 차이를 최소화하는 선 찾기
- 즉, **Loss Function**을 최소로 만드는 β_0 과 β_1 찾기

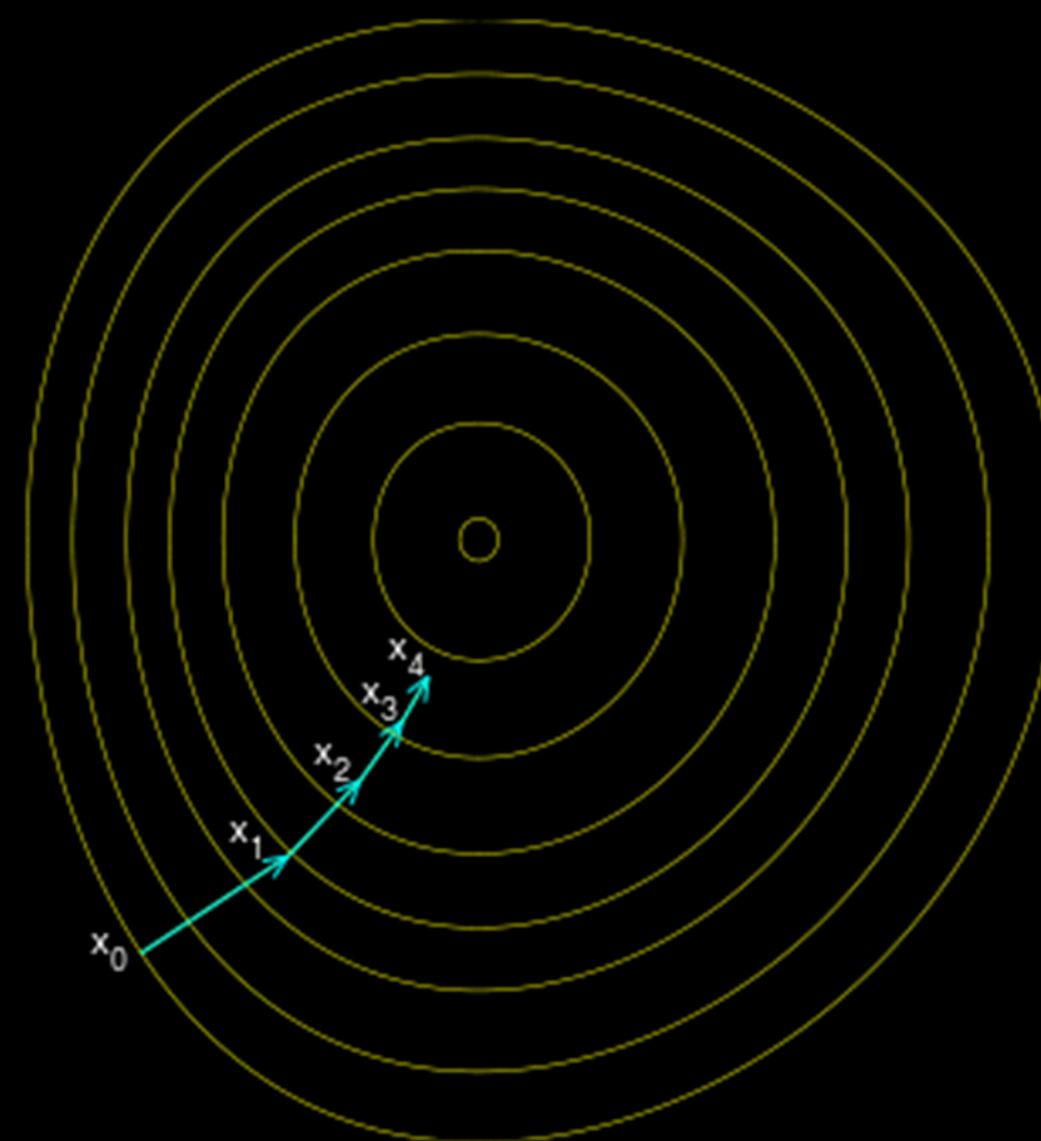




- 손실함수 (Loss Function)
- 예측 값과 실제 값 간의 오차
- 손실함수를 최소화하기 위한 모델의 인자를 찾는 과정 필요



- 산 정상 오르기
- 산 정상이 되는 지점을 찾고 싶다.
- 아무 곳에서나 시작했을 때, **가장 정상을 빠르게 찾아가는 방법**은 무엇일까?



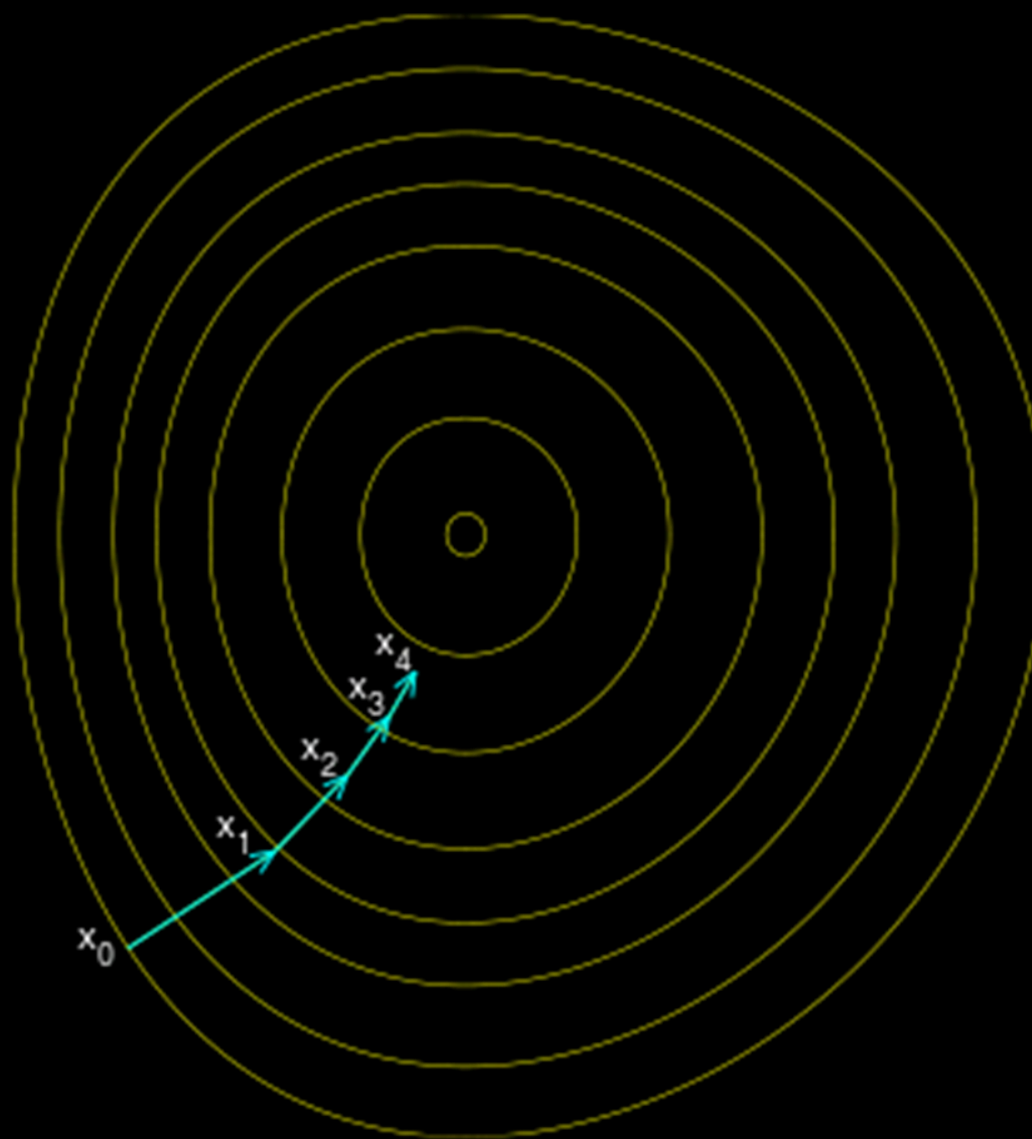


- 가정

- 정상의 위치는 알 수 없다.
- 현재 나의 위치와 높이는 알 수 없다.
- 내 위치에서 일정 수준 이동할 수 있다.

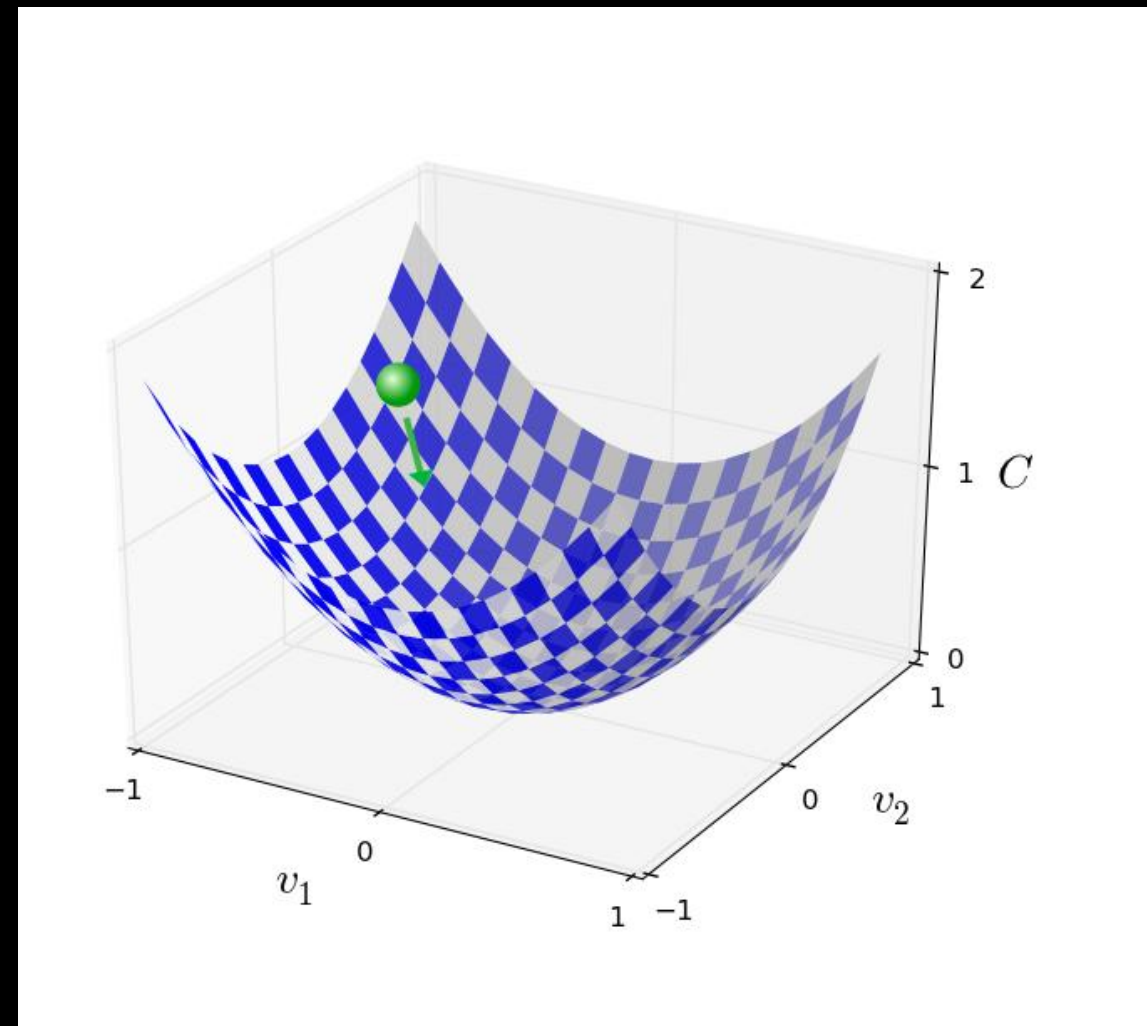
- 방법

- 현재 위치에서 **가장 경사가 높은 쪽**을 찾는다.
- 오르막 방향으로 **일정 수준 이동**한다.
- 더 이상 높이 변화가 없을 때까지** 반복한다.





- Gradient Descent (경사하강법)
- 최적의 값을 찾기 위한 거꾸로 된 산을 내려가는 방법
- **Loss function**을 최소로 만드는 β_0 과 β_1 선정





Loss Function을 최소화하는 Gradient Descent를 통해 데이터를 가장 잘 설명할 수 있는 선을 찾는 방법

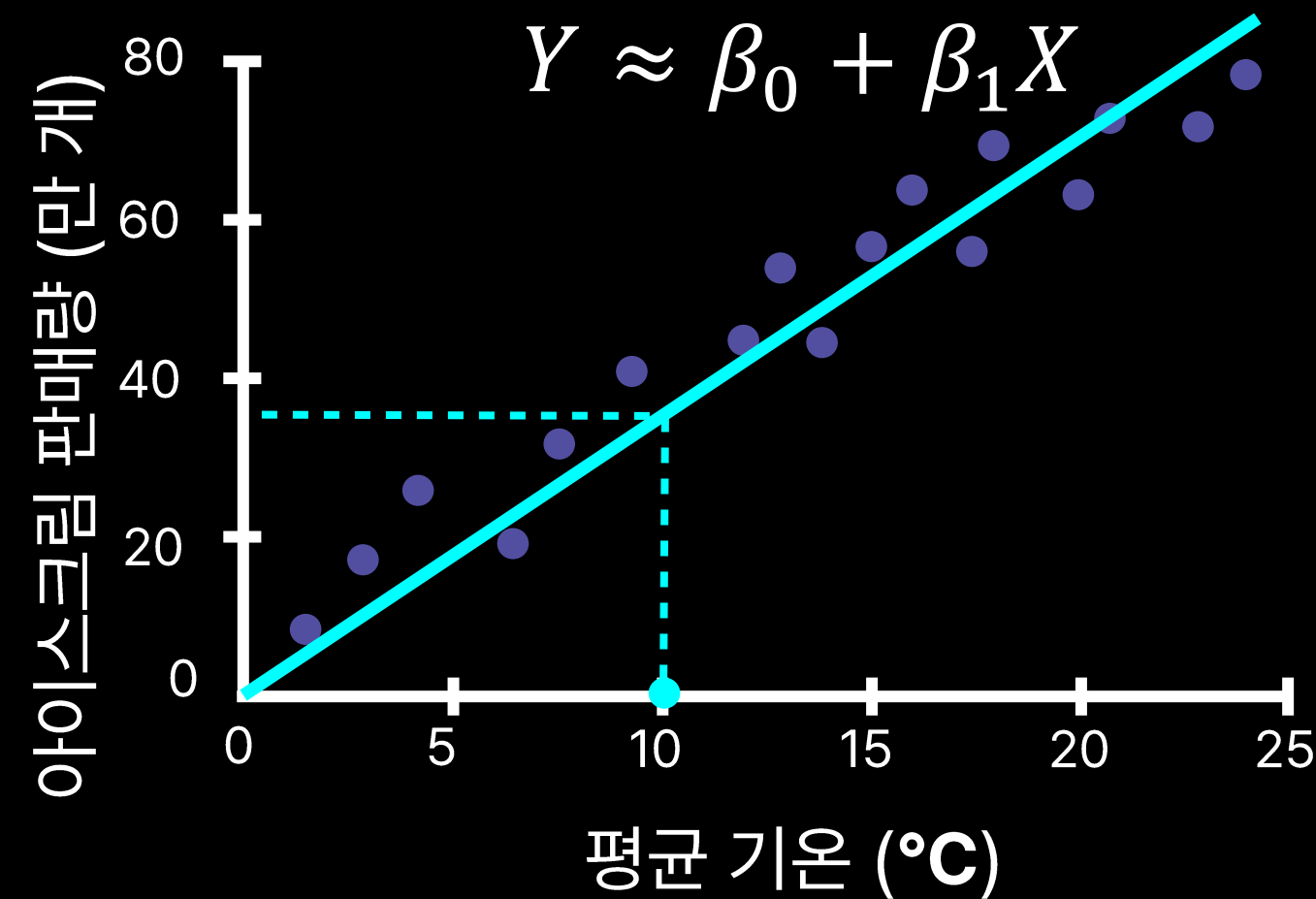
- Loss Function
 - 실제 값과 모델이 예측하는 값의 오차
- Gradient Descent
 - 최적의 β_0 과 β_1 을 찾는 알고리즘

2. 단순 선형 회귀



- 평균 기온에 따른 아이스크림 판매량을 예측하고자 함
- $Y \approx \beta_0 + \beta_1 X$
- 아이스크림 판매량 = $\beta_0 + \beta_1$ * 평균 기온





- 데이터를 잘 설명할 수 있는 선을 찾아 새로운 데이터에 대한 결과값 확인 가능
- 이러한 가장 단순한 회귀 모델을 **단순 선형 회귀(Simple Linear Regression)**라고 함



$$Y \approx \beta_0 + \beta_1 X$$

- 가장 기본적이고 간단한 방법의 회귀 알고리즘
- 입력값 X 와 결과값 Y 의 관계를 설명할 때 가장 많이 사용되는 단순한 모델
- 회귀 알고리즘의 기초로 이를 응용한 다수의 알고리즘 존재



$$Y \approx \beta_0 + \beta_1 X$$

- 가장 기초적이나 여전히 많이 사용되는 알고리즘
- 입력 값 X 가 1개인 경우에만 적용 가능
- 입력 값이 결과 값에 얼마나 영향을 미치는지 알 수 있음
- 두 변수 간의 관계를 직관적으로 해석하고자 하는 경우 활용



만약, 평균 기온 뿐만 아니라 강수량이라는 **새로운 입력값을 추가**하여
판매량을 예측하고자 한다면 어떻게 해야 할까?

- **단순 선형 회귀를 응용한 회귀 알고리즘**
- 데이터의 유형 및 예측 결과에 따라 **다양한 회귀 알고리즘** 필요

3. 다중 선형 회귀와 다항 회귀

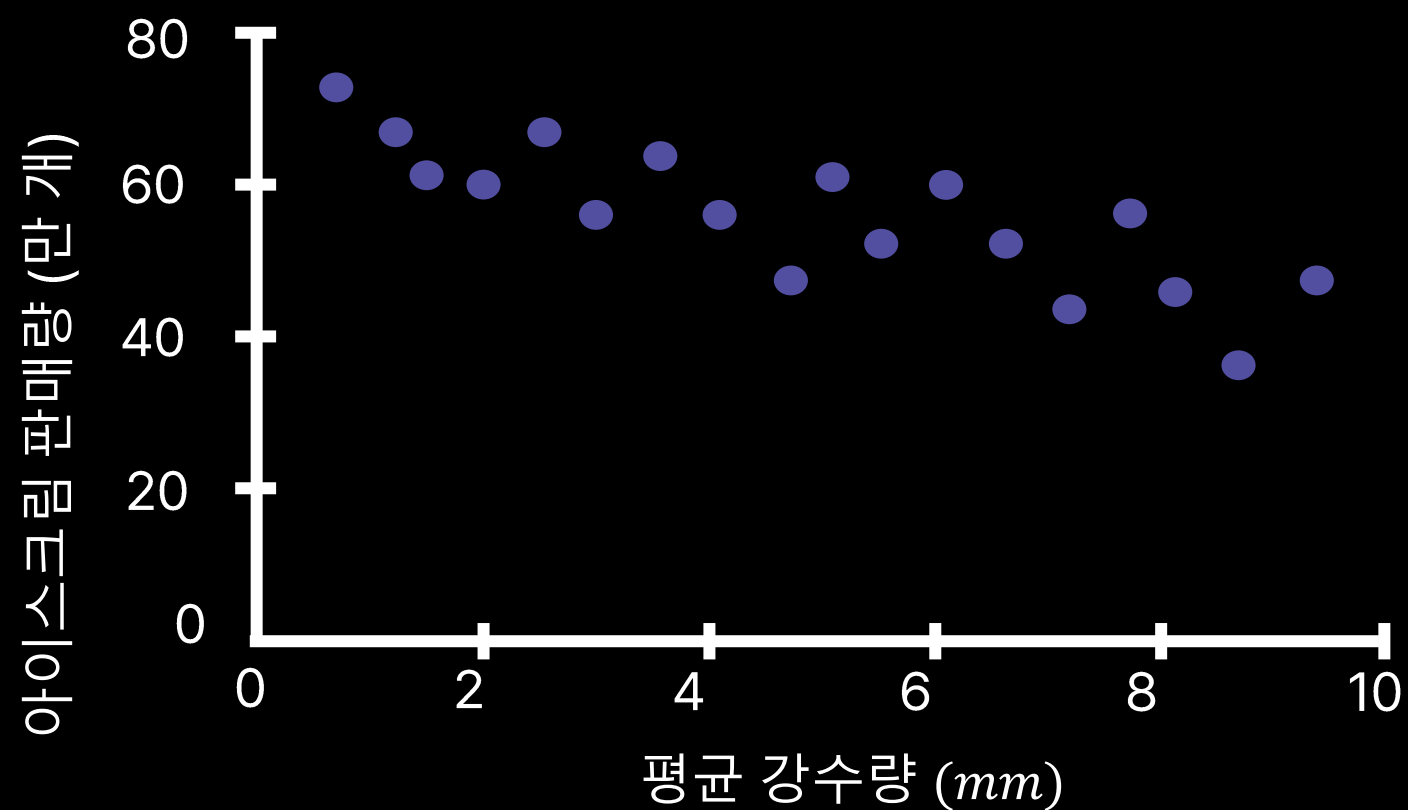


- 평균 기온에 따른 아이스크림 판매량을 예측하고자 함
- $Y \approx \beta_0 + \beta_1 X$
- 아이스크림 판매량 = $\beta_0 + \beta_1$ * 평균 기온





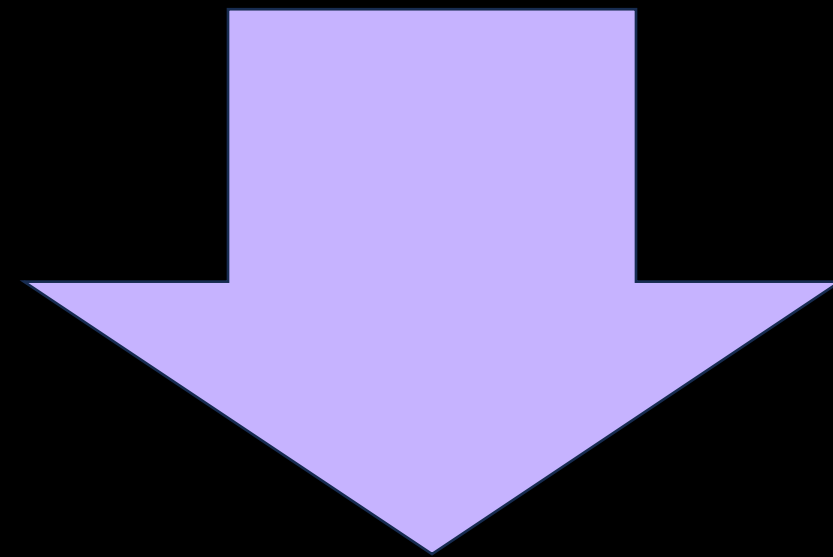
- 만약 입력값 X 에 강수량이 추가된다면?
- 즉, X 에 평균 기온과 평균 강수량에 따른 Y 아이스크림 판매량을 예측하고자 할 때



X_0	X_1	Y
평균 기온(°C)	평균 강수량(mm)	아이스크림 판매량(만개)
10	10	40
13	7.5	52.3
20	2	60.5
25	0	80



- X
평균 기온과 평균 강수량에 따른 Y
아이스크림 판매량을 예측하고자 할 때
- 여러 개의 입력값 (X_i)으로 결과값(Y)을 예측하고자 하는 경우



다중 선형 회귀 (Multiple Linear Regression)



$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_i X_i$$

- **입력값 X 가 여러 개**(2개 이상)인 경우 활용할 수 있는 회귀 알고리즘
- 각 개별 X_i 에 해당하는 **최적의 β_i** 를 찾아야 함
- 문제 : 평균 기온과 평균 강수량에 따른 아이스크림 판매량 예측
 - 아이스크림 판매량 = $\beta_0 + \beta_1 * \text{평균 기온} + \beta_2 * \text{강수량}$

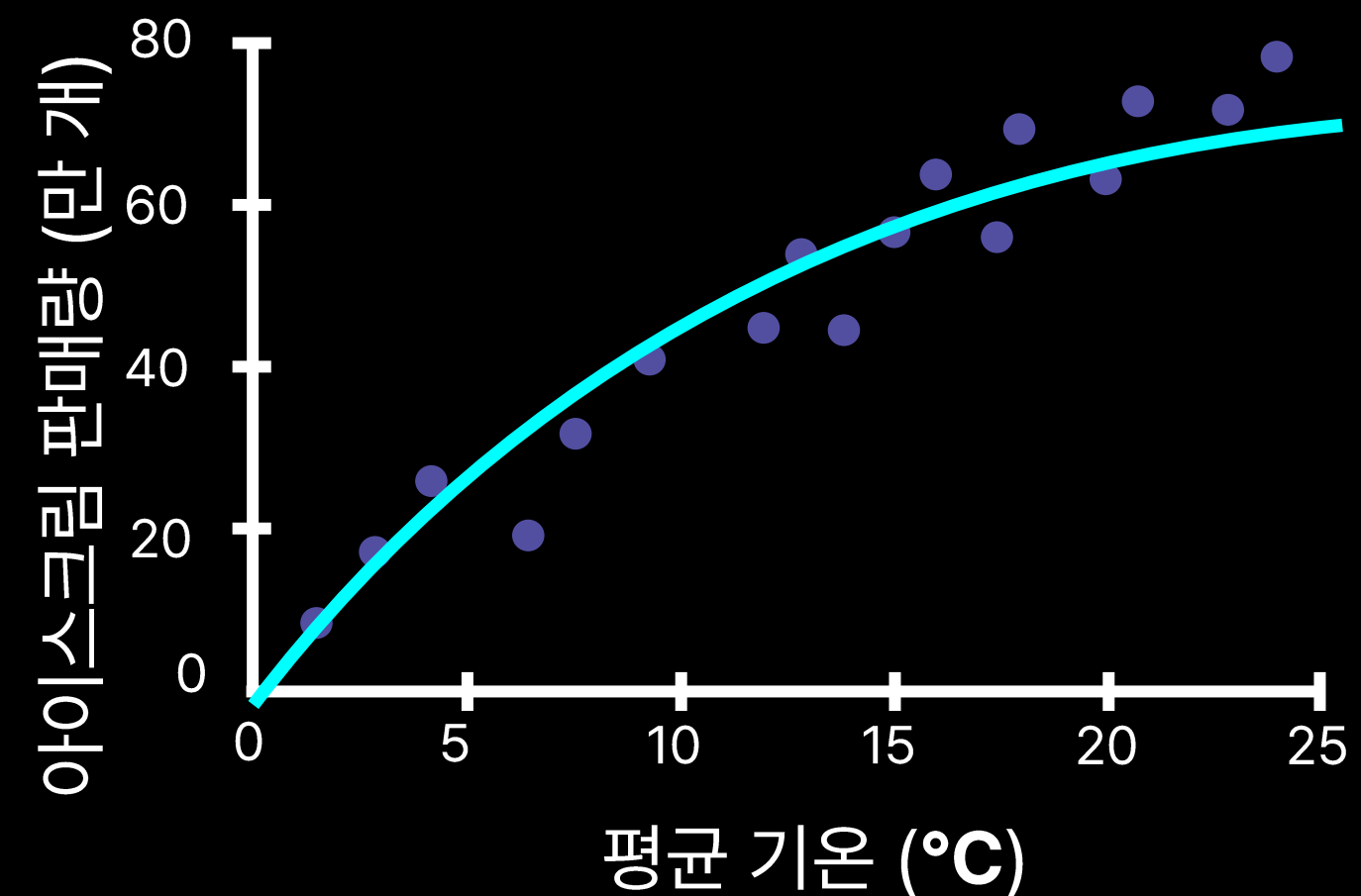
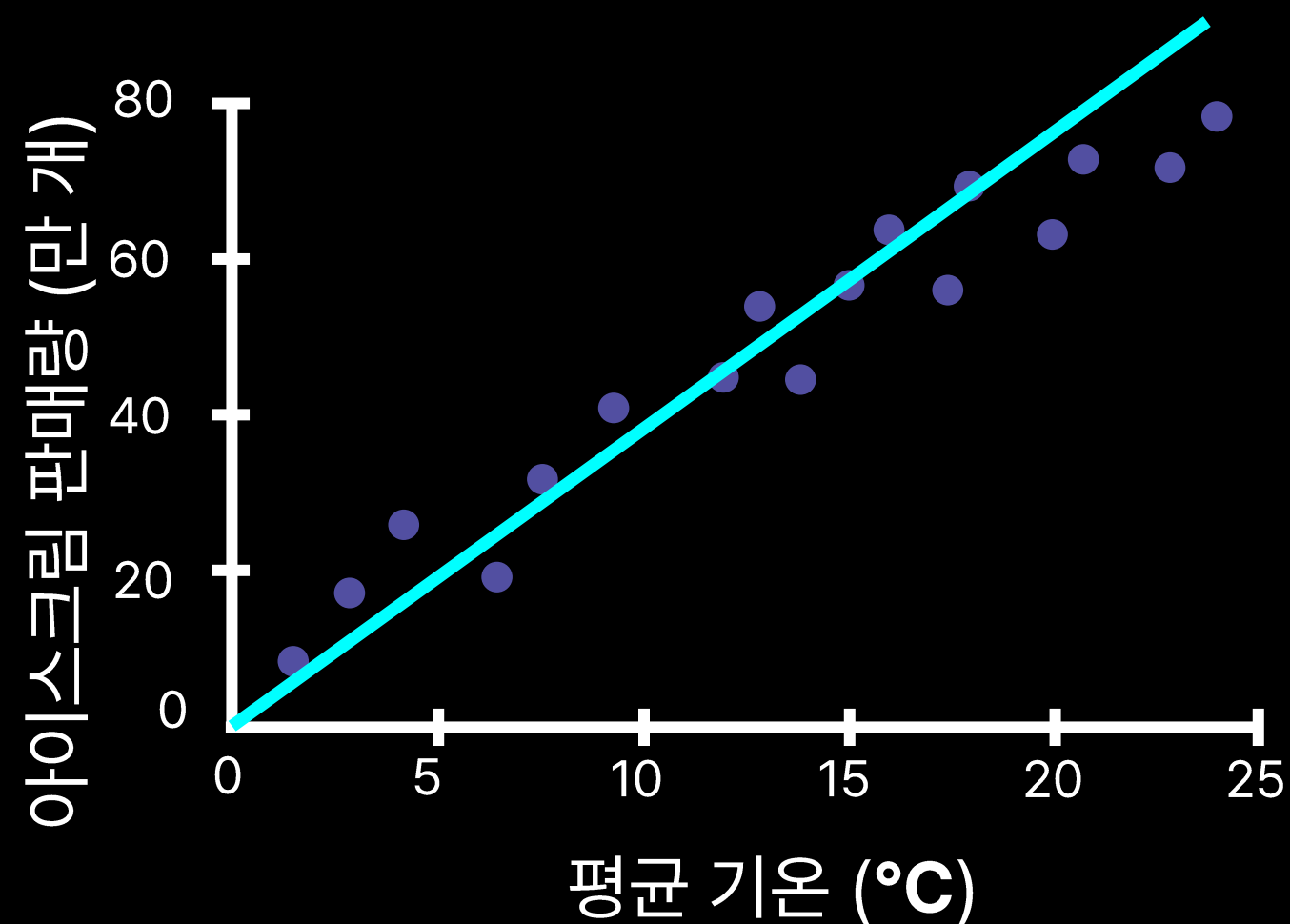


$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_i X_i$$

- 여러 개의 입력 값과 결과 값 간의 관계 확인 가능
- 어떤 입력 값이 결과값에 어떤 영향을 미치는지 알 수 있음
- 여러 개의 입력 값 사이 간의 **상관관계가 강할 경우 결과에 대한 신뢰성을 잃을 가능성**이 있음
 - 상관관계 : 두 가지 변수 중 한 변수가 변화하면 다른 한 변수도 따라서 변화하는 관계

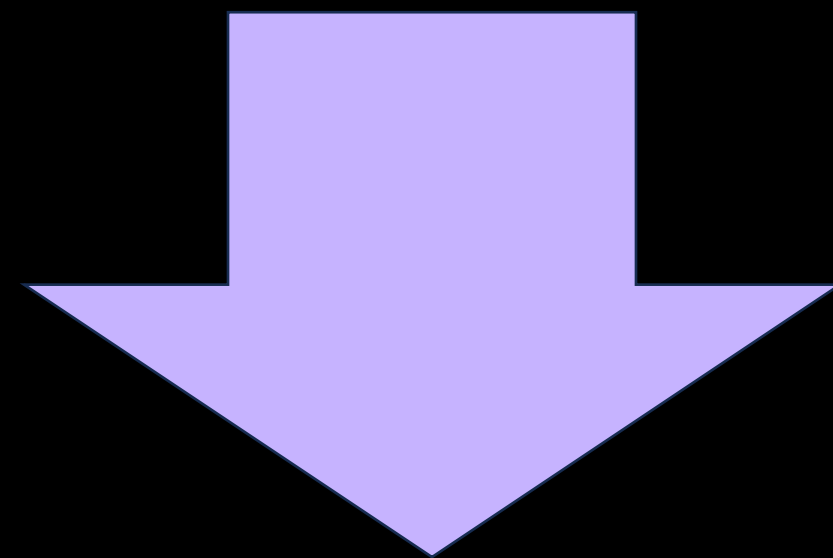


다중 선형 회귀 알고리즘 적용 시, 예측 결과값이 좋지 않은 경우
만약, 평균기온과 예측 판매량 간의 관계가 **비선형적**이라면?





다중 선형 회귀 적용 시 예측 결과가 좋지 않고,
변수들 간의 관계가 선형적이지 않은 경우



다항 회귀

(Polynomial Linear Regression)

$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2^2 + \dots + \beta_i X_i^i$$

아이스크림 판매량 = $\beta_0 + \beta_1 * \text{평균기온} + \beta_2 * \text{강수량}$

- 1차 함수 선형식으로 표현하기 어려운 분포의 데이터를 위한 회귀
- **복잡한 분포의 데이터**의 경우, 일반 선형회귀 알고리즘 적용 시 낮은 성능의 결과 도출



$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2^2 + \dots + \beta_i X_i^i$$

- 일차 함수 식으로 표현할 수 없는 복잡한 데이터 분포에도 적용 가능
- 극단적으로 높은 차수의 모델을 구현할 경우 과적합 현상 발생
 - 과적합 : 모델이 과도하게 학습 데이터에 맞춰지는 현상
- 데이터 관계를 선형으로 표현하기 힘든 경우 사용



```
import pandas as pd
from sklearn.model_selection import train_test_split

# 데이터셋 불러오기
df = pd.read_csv('data.csv')

train, test = train_test_split(df, test_size=0.2, random_state=21)
```

- **train_test_split()**

- 데이터 세트를 분리해주는 함수
- *arrays : 데이터셋
- test_size : 테스트 세트의 비율 (0~1)
- random_state : 분할 과정에서의 무작위성 제어



```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
np.random.seed(0)
```

```
X = 5*np.random.rand(100,1)
```

```
y = 3*X + 5*np.random.rand(100,1)
```

데이터 분리

```
train_X, test_X, train_y, test_y = train_test_split(X, y, test_size = 0.3, random_state = 0)
```

```
simpleLinear = LinearRegression()
```

모델 생성

```
simpleLinear.fit(train_X, train_y)
```

모델 학습

```
predicted = simpleLinear.predict(test_X)
```

테스트 데이터에 대한 예측

```
model_score = simpleLinear.score(test_X, test_y)
```

모델을 평가한 점수 확인

```
beta_0 = simpleLinear.intercept_
```

학습이 완료된 모델의 β_0 반환

```
beta_1 = simpleLinear.coef_
```

학습이 완료된 모델의 β_1 반환



다항 회귀의 입력값을 변환하기 위한 모듈을 불러옵니다.

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly_feat = PolynomialFeatures(degree=2, include_bias = True)
```

```
poly_X = poly_feat.fit_transform(X)
```

```
multilinear = LinearRegression()
```

```
multilinear.fit(poly_x, y)
```

- 다항 회귀는 먼저 **입력 데이터 X에 대해 전처리를 진행한 후** 다중 선형 회귀를 진행
- 전처리한 데이터를 다중 선형 회귀에 적용했던 방식과 동일하게 진행



다항 회귀의 입력값을 변환하기 위한 모듈을 불러옵니다.

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly_feat = PolynomialFeatures(degree=2, include_bias = True)
```

```
poly_X = poly_feat.fit_transform(X)
```

```
multilinear = LinearRegression()
```

```
multilinear.fit(poly_x, y)
```

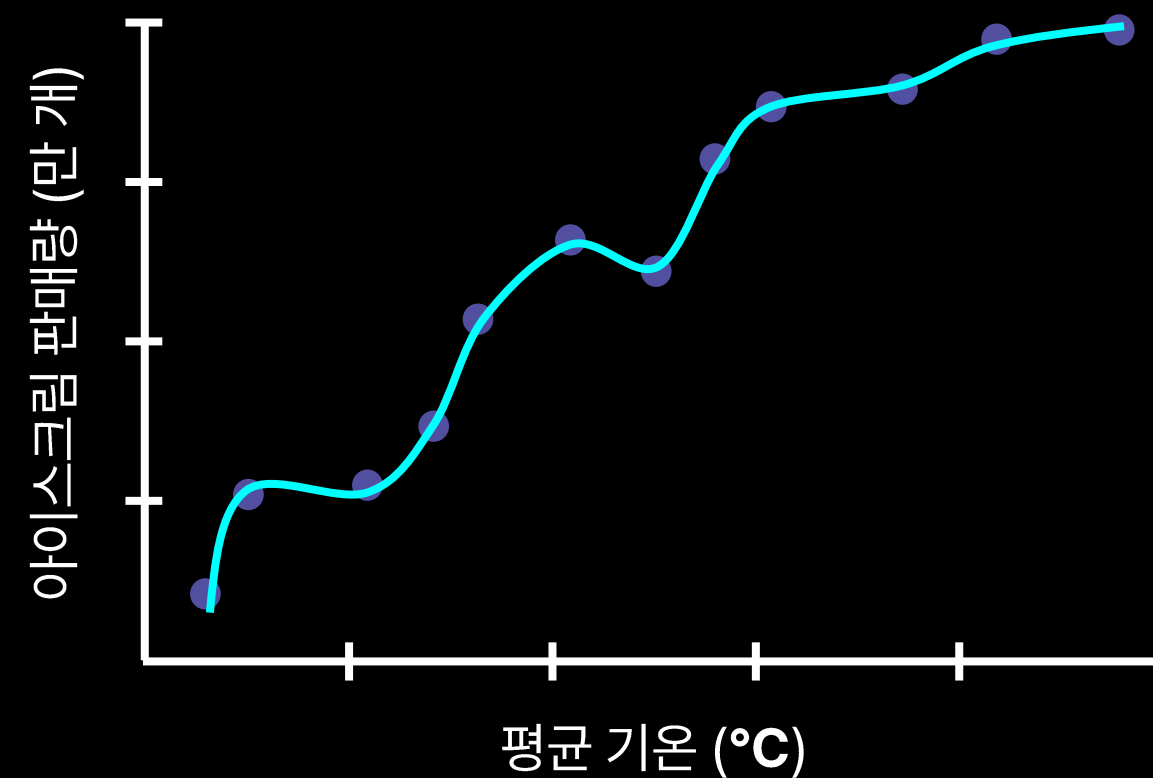
- degree = 다항식의 차수
- include_bias = 편향 변수 추가 여부 (다항식의 모든 거듭제곱이 0일 경우 편향 변수가 추가됨)
- fit_transform() : X와 X의 degree 제곱을 추가한 데이터 반환

4. 과적합과 정규화

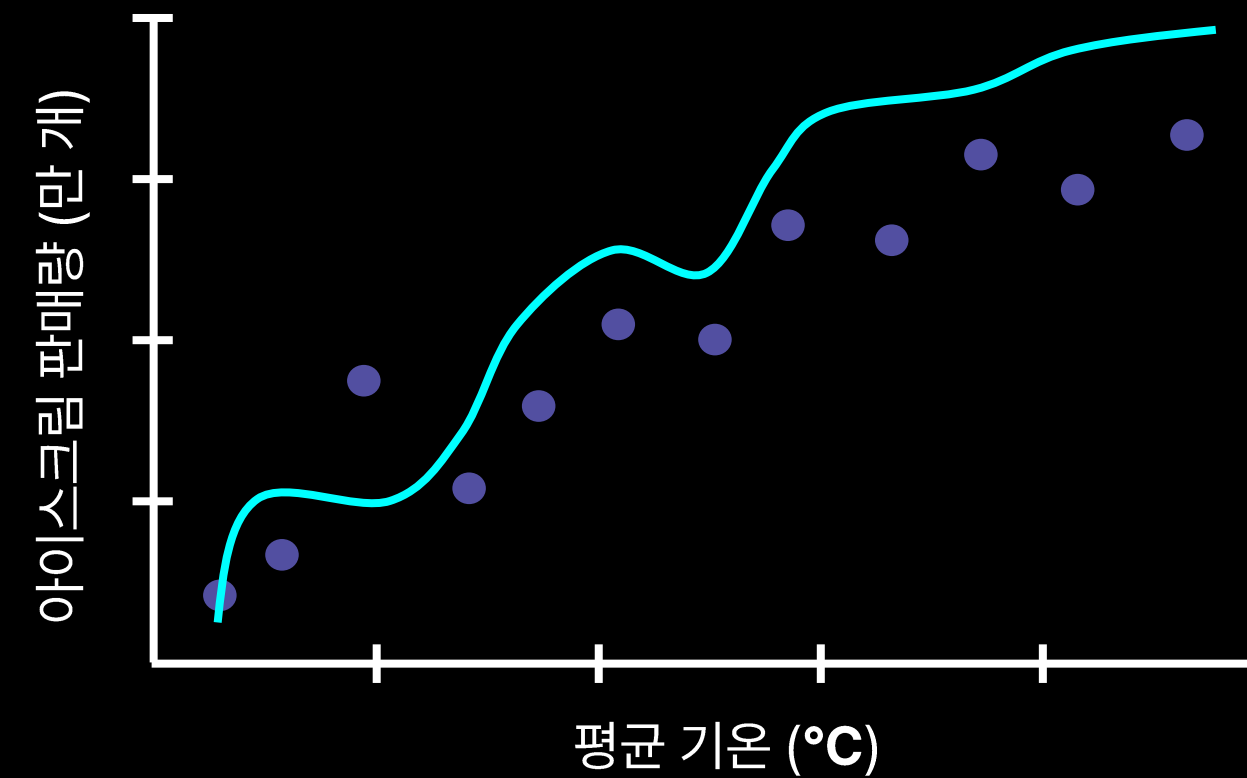
과거 데이터를 학습한 다항 회귀 모델이 새로운 데이터에 대한 예측 결과가 매우 낮다면?

→ 과적합 발생(Overfitting)

2019년 데이터로 학습한 다항 회귀 모델 A



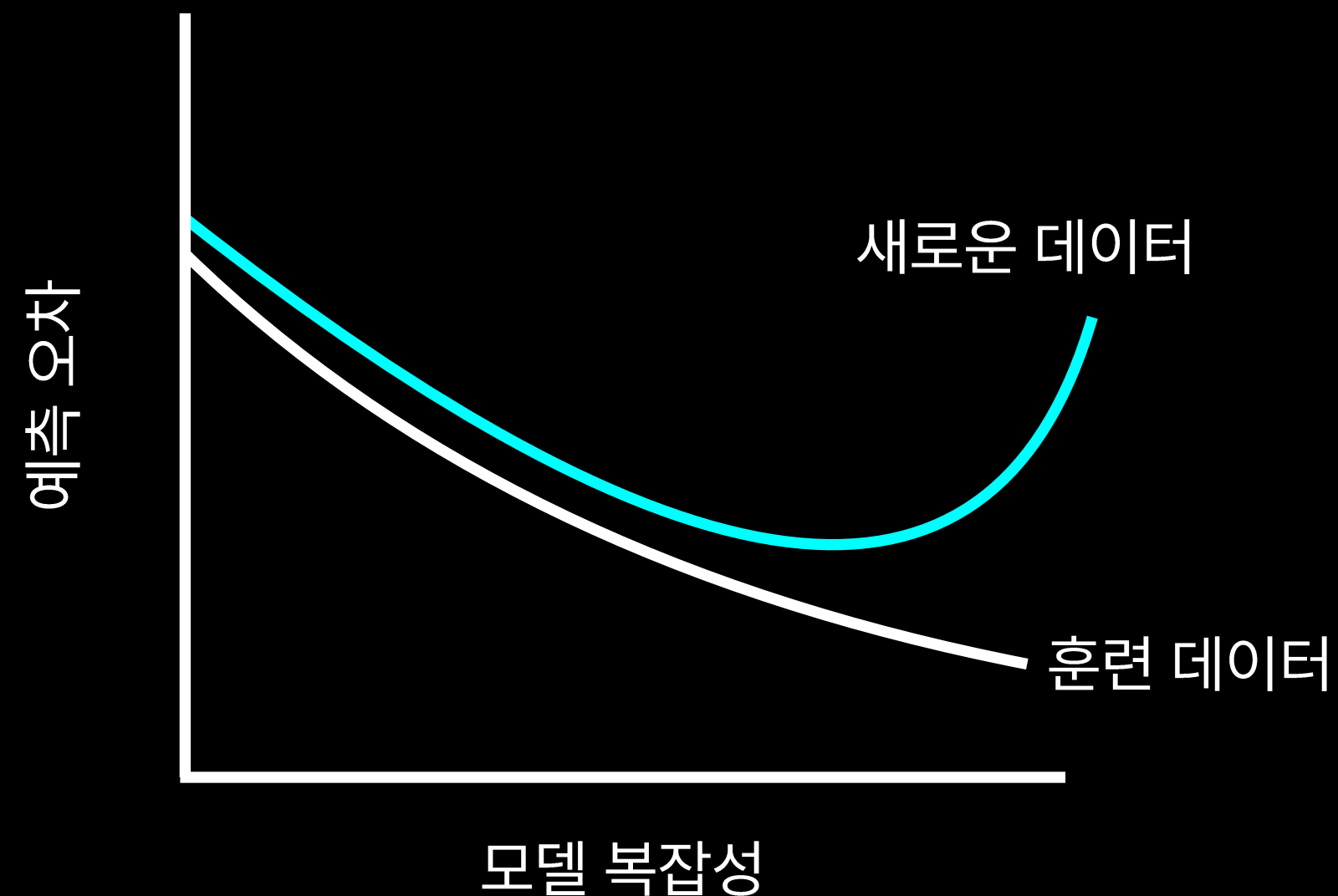
학습된 다항 회귀 모델 A에 새로운 데이터 입력





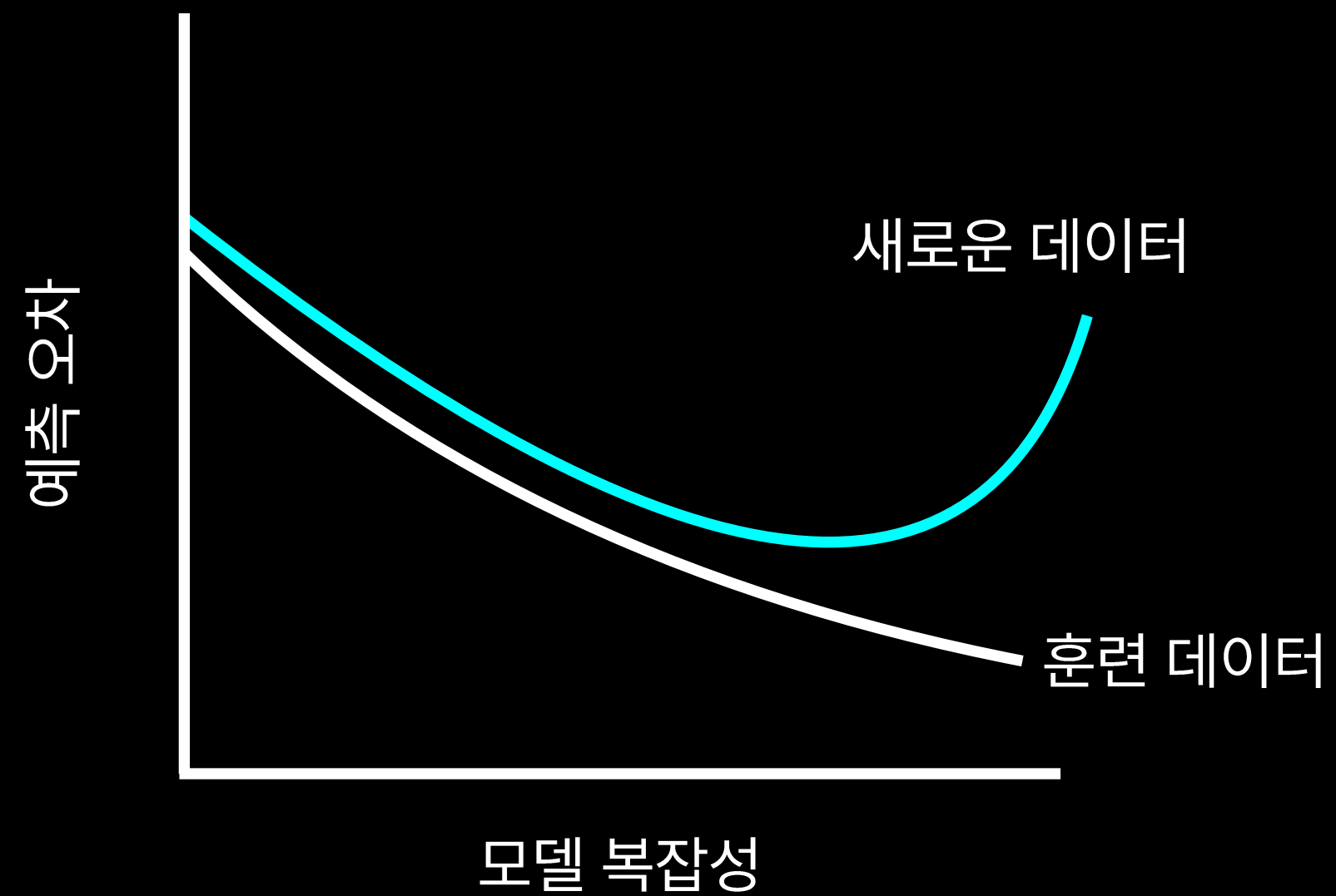
모델이 주어진 훈련 데이터에 과도하게 맞춰져 **새로운 데이터가 입력** 되었을 때,
잘 예측하지 못하는 현상

- 즉, 모델이 과도하게 복잡해져 일반성이 떨어진 경우를 의미함





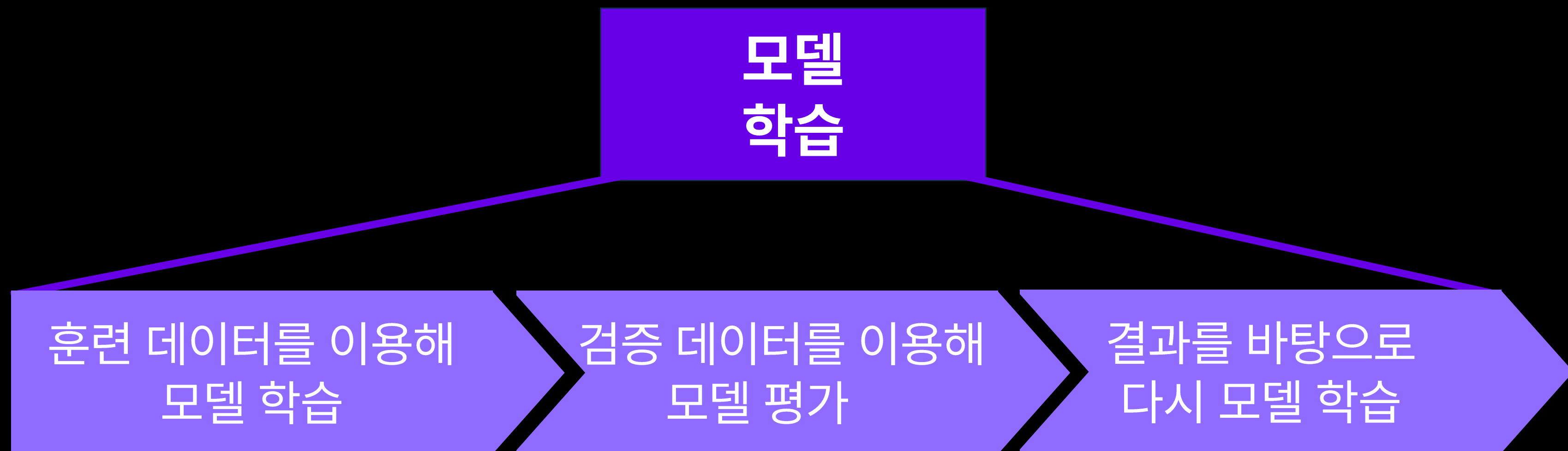
- 모델이 실제 데이터에서도 잘 적용될 수 있도록 **과적합 방지를 위한 다양한 방법** 사용
 - 검증 과정
 - 정규화





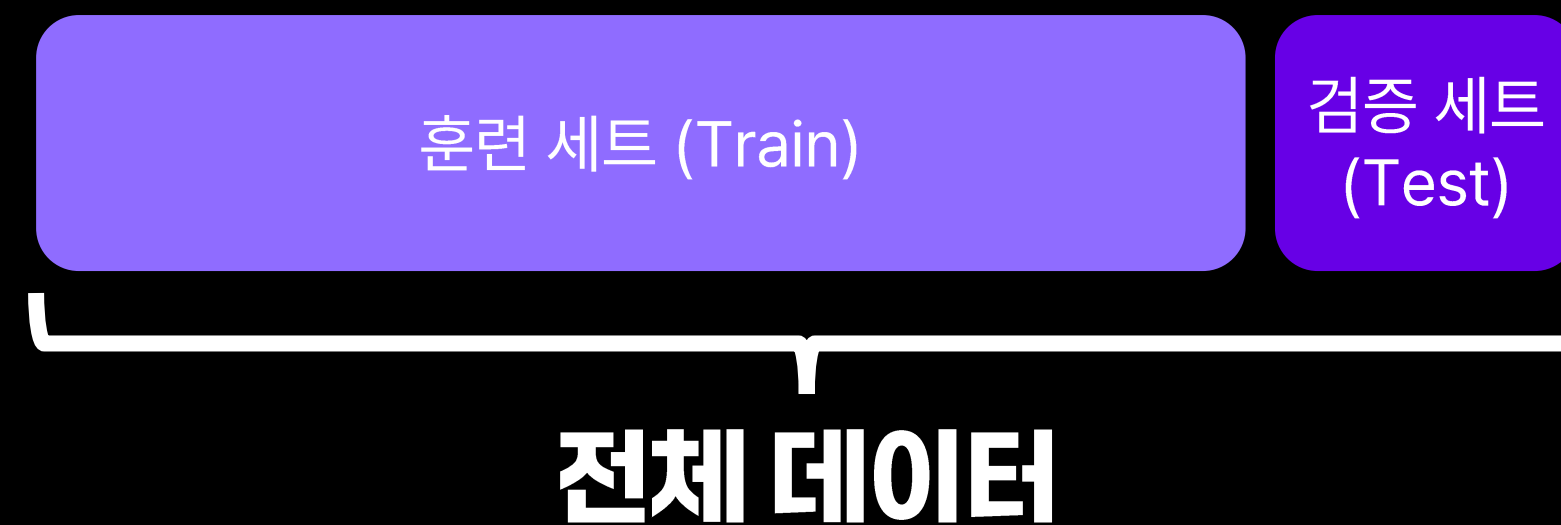
• 검증 과정

- 훈련 데이터로 훈련 시킨 모델의 일반화 능력을 평가하는 과정
- 테스트 과정 이전에 진행되는 단계



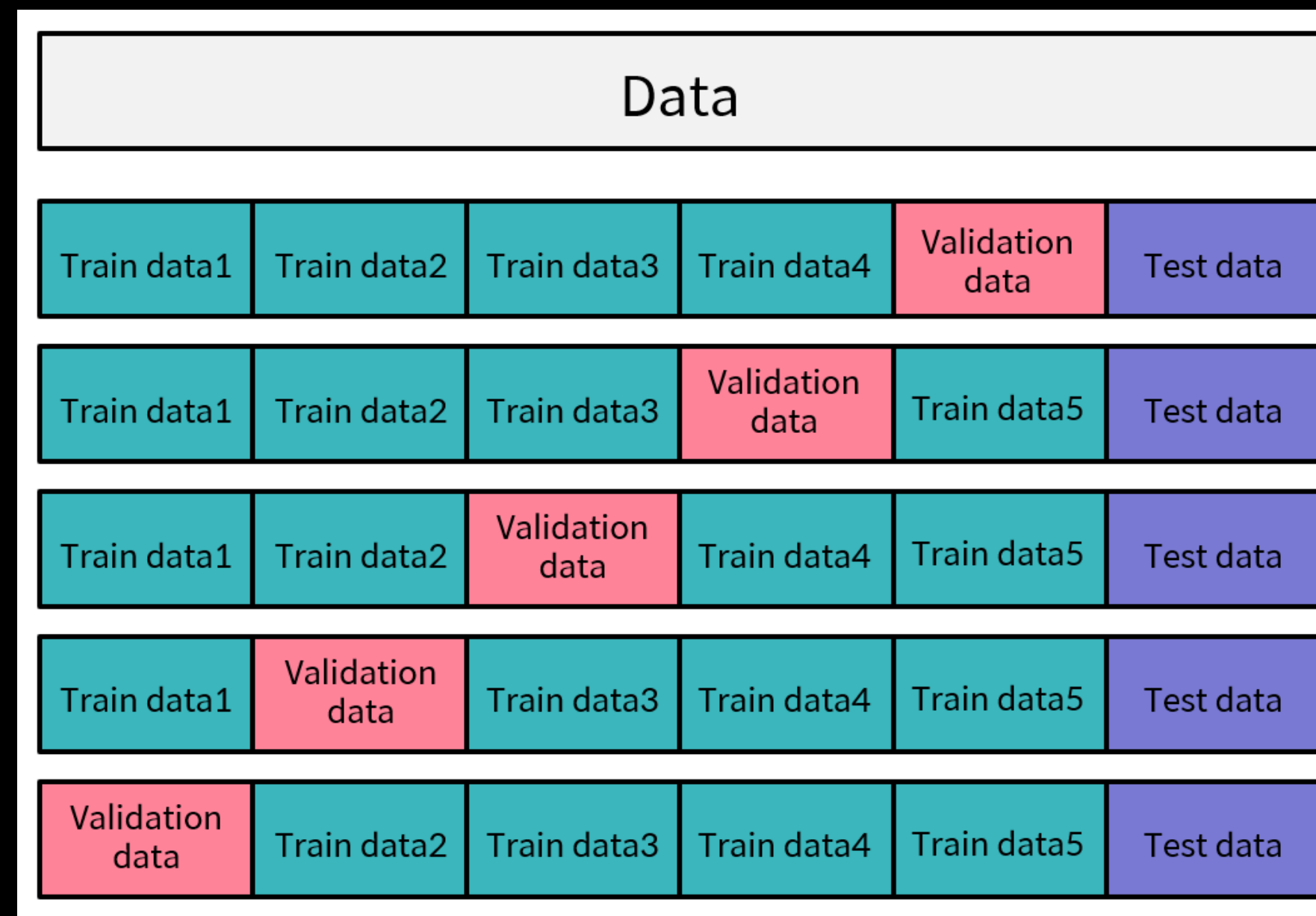


- 기본적인 검증 방식으로, 학습에 사용되지 않는 검증 세트로 두어 검증 진행
 1. **훈련**세트를 학습시켜 모델 생성
 2. **검증**세트로 모델의 성능 확인
 3. 모델의 **하이퍼파라미터** 조정 또는 모델 변경
 1. **하이퍼파라미터**: 모델을 구성하진 않지만, 모델의 학습에 관여하는 조정 가능한 값 (예시: 학습률)
 4. 기대 성능에 도달할 때까지 위 단계를 반복





- 제한된 데이터 샘플에서 머신러닝 모델을 평가하는데 사용하는 리샘플링 절차
- 학습에 사용된 데이터와 검증 데이터에 대한 모델의 예상 적합도 수준 (성능) 을 추정
- 이를 위해 서로 다른 데이터 세트로 여러 번 나눈 뒤 모델을 검증하는 방식.



K-Fold 교차 검증



- K-fold 교차 검증
 - 훈련 데이터를 계속 변경하며 모델을 훈련 시킴
 - 데이터를 K등분으로 나누고 K번 훈련
- K-fold 교차 검증을 이용한 모델 검증 순서
 1. Fold를 나누는 기준인 K를 설정하여 데이터셋을 K 그룹으로 나눔
 2. K개 중 한 개를 검증용, 나머지 (K-1) 개를 훈련용으로 사용
 3. K개 모델의 평균 성능이 최종 모델 성능



```
from sklearn.model_selection import KFold
```

```
kfold = KFold(n_splits=5)
```

 교차 검증을 위한 객체 생성

```
for train_idx, val_idx in kfold.split(train_X):
```

 train과 test로 분리된 **인덱스**를 반환

```
    X_train, X_val = train_X[train_idx], train_X[val_idx]  
    y_train, y_val = train_y[train_idx], train_y[val_idx]
```

```
    # 동일한 가중치 사용을 위해 각 fold 마다 모델 초기화 하기  
    model = LinearRegression()
```

```
    model.fit(X_train, y_train)
```

```
    # 각 Iter 별 모델 평가 점수 측정  
    score = model.score(X_val, y_val)
```

- `n_splits` = 분리할 데이터(fold) 개수

5. 정규화를 적용한 회귀



- 학습 데이터에만 너무 특화되지 않고 일반화가 가능하도록 일종의 규제를 가하는 기법
- 모델의 복잡성을 줄여 일반화된 모델 구현을 위한 방법
- 대표적으로 L1 정규화 (Lasso), L2 정규화 (Ridge), 엘라스틱 넷 (Elastic net)이 있음

방법	의미
L1 정규화 (Lasso)	• 정말 중요하지 않은 정보는 아예 무시하도록 하는 방법 • 불필요한 정보를 제거하여 모델을 단순화 시키고, 중요한 정보에 집중하도록 한다. • 해석이 쉬운 모델이 필요하거나 일부 특성만 필요할 때 유용
L2 정규화 (Ridge)	• 복잡하게 생각하지 말고, 가능한 단순하게 문제를 풀도록 하는 방법 • 모델이 너무 복잡해지는 것을 방지하고, 새로운 문제에도 잘 대응할 수 있다. • 모든 변수를 사용하는 것이 유리하거나 변수 간의 상관관계가 높을 때 유용
엘라스틱 넷 (Elastic net)	• 복잡하게 생각하지 말고, 단순하게 문제를 풀되, 정말 중요하지 않은 정보는 무시하도록 하는 방법 • Ridge와 Lasso의 장점을 결합한 방법 • 모델이 데이터를 잘 이해하도록 도와주면서, 동시에 중요하지 않은 정보는 무시하게 만든다. • L1 정규화와 L2 정규화를 모두 활용하고 싶거나, 어느 쪽이 더 적합한지 모를 때 유용

- L1 정규화 (Lasso Regularization)
 - 모델이 과적합 되지 않도록 손실함수에 L1 규제를 더하여 페널티를 부과하는 방법
 - 불필요한 입력 값에 대응되는 β_i 를 정확히 0으로 만듦
 - 몇 개의 중요한 가중치들만 남음

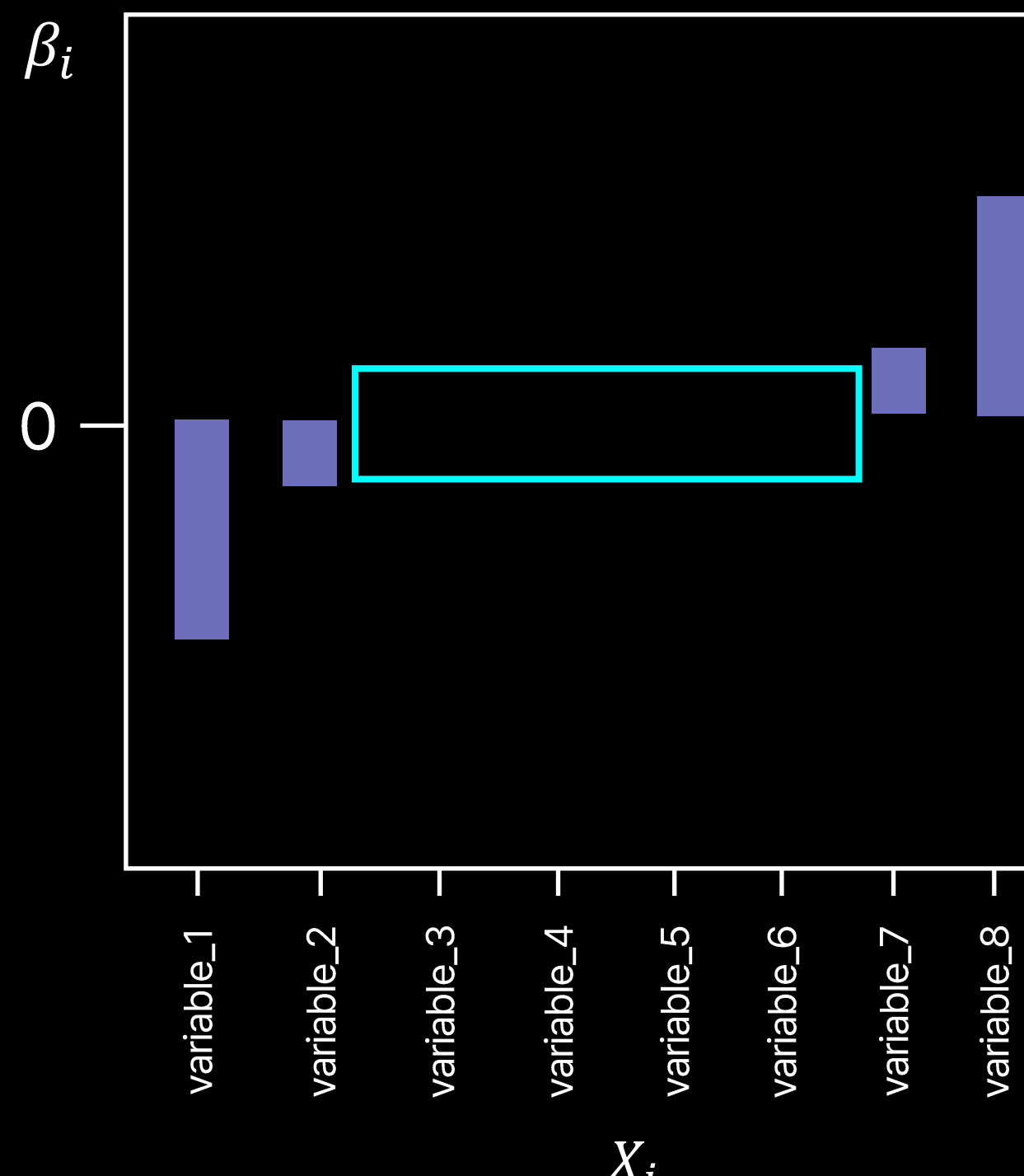
$$L1 = Loss + \alpha \sum_j |w_j|^1$$

정규화 부분 (L1 규제)

$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2^2 + \cdots + \beta_i X_i^i$$



- 너무 많은 β_i 를 0으로 만들 수 있어 모델의 정확성이 떨어질 수 있음
- **특성 선택이 필요한 경우** 유용
- 몇 개의 중요 변수만 선택하기 때문에 **정보 손실의 가능성**이 있음



- L2 정규화 (Ridge Regularization)
 - 모델이 과적합 되지 않도록 손실함수에 L2 규제를 더하여 페널티를 부과하는 방법
 - 아주 큰 값이나 작은 값을 가지는 이상치에 대한 β_i 를 0에 가까운 값으로 만듦
 - L1 정규화에 비해 0으로 수렴하는 가중치의 개수가 적음

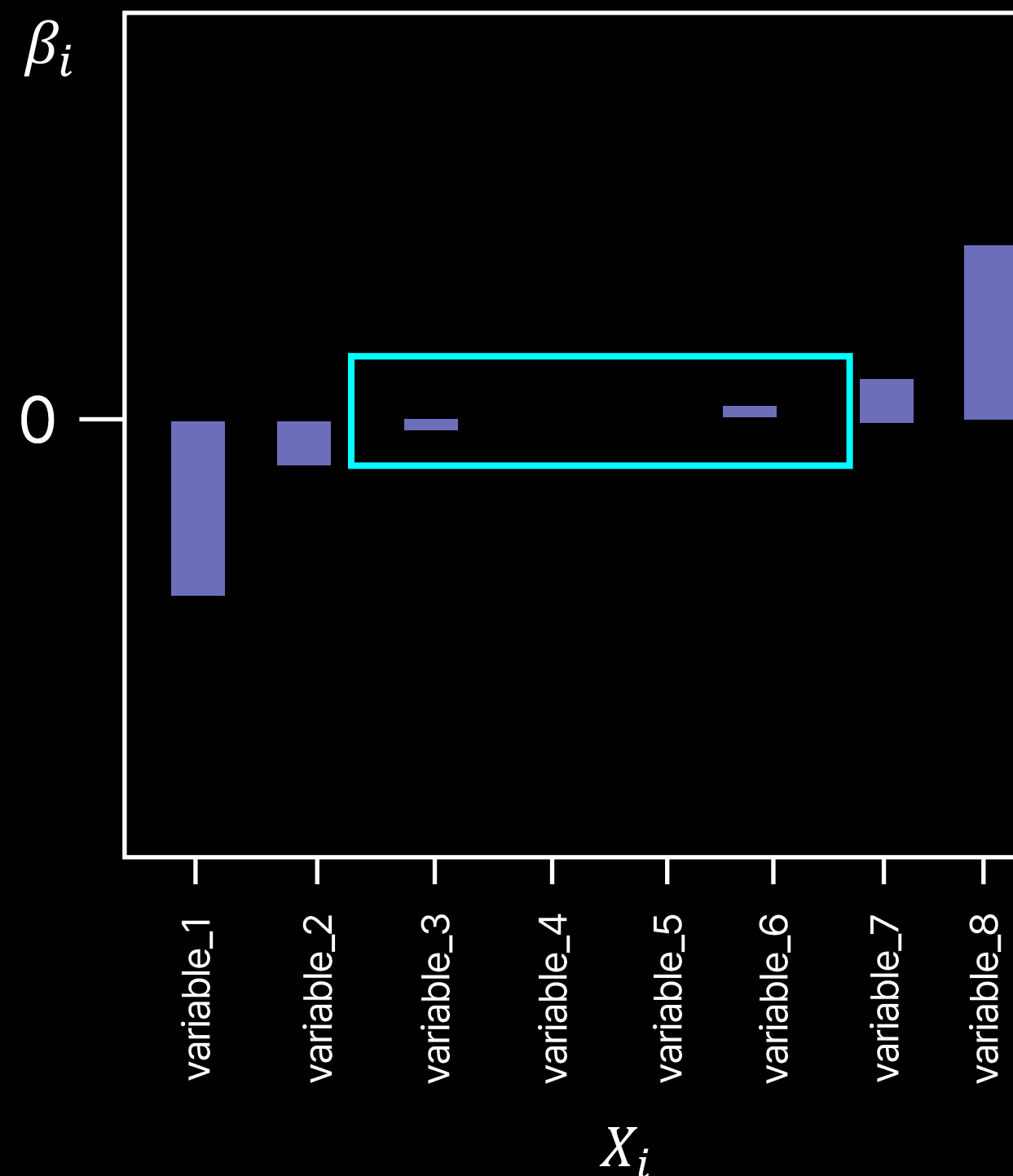
$$L2 = Loss + \alpha \sum_j |w_j|^2$$

정규화 부분 (L2 규제)

$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2^2 + \cdots + \beta_i X_i^i$$



- β_i 를 0에 가깝게 만들지만 완전한 0은 아니기 때문에 모델이 여전히 복잡할 수 있음
- **특성들 간의 강한 상관관계를 가질 때** 유용하게 사용됨



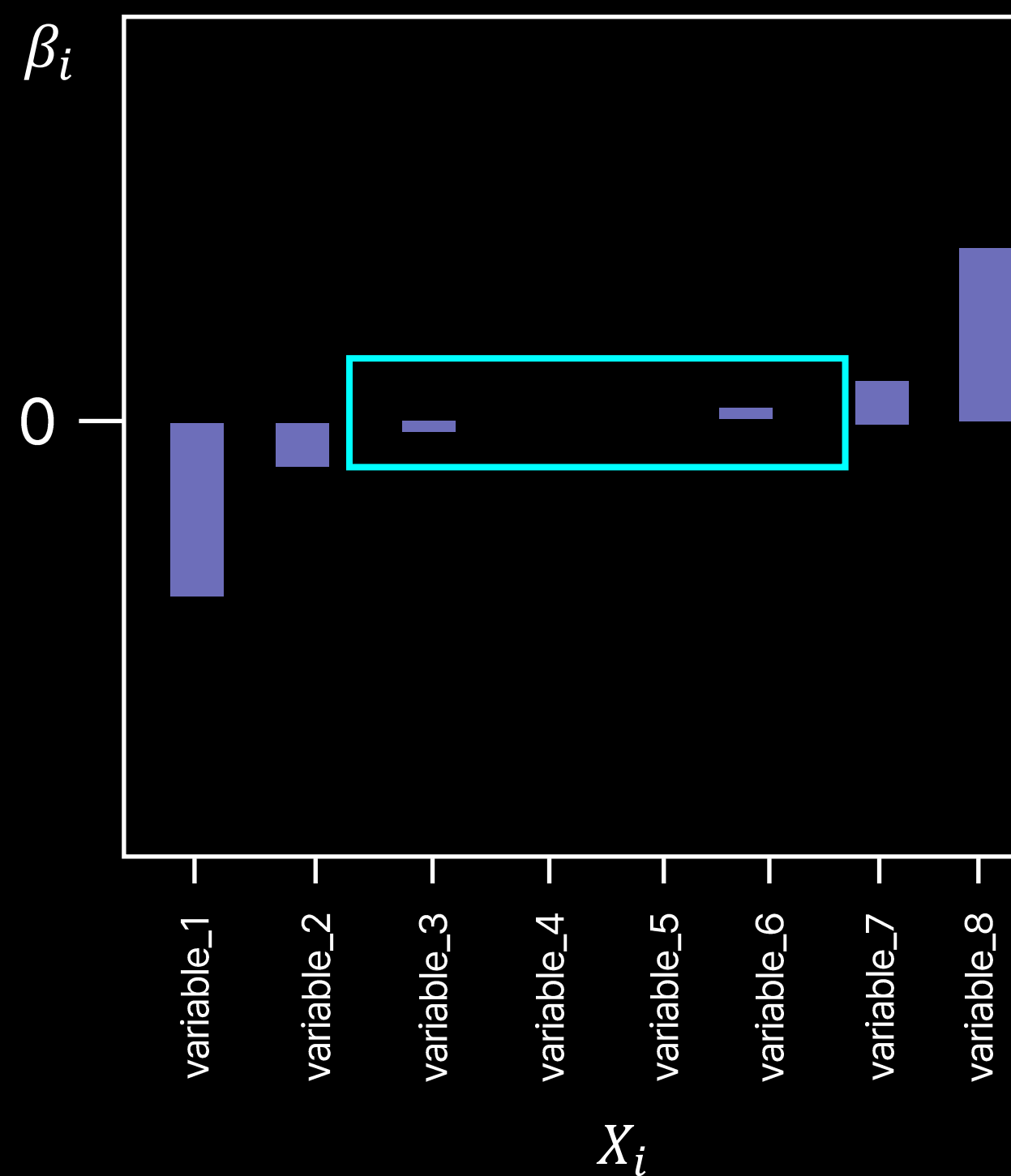


- L1 정규화와 L2 정규화 적용 비율을 조정한 회귀
 - α : 정규화 적용 강도, α 값이 높을 수록 정규화가 강하게 적용된다.
 - γ : L1 정규화 적용 비율 (0 이상 1 이하)

$$Elastic = Loss + \alpha \left(\underbrace{\gamma \sum_j |w_j|^1}_{\text{정규화 부분 (L1 규제)}} + \underbrace{(1 - \gamma) \sum_j |w_j|^2}_{\text{정규화 부분 (L2 규제)}} \right)$$



L1 정규화와 L2 정규화 적용 비율을 조정한 회귀





```
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso

ridge_reg = Ridge(alpha = 10)
ridge_reg.fit(train_X, train_y)

lasso_reg = Lasso(alpha = 10)
lasso_reg.fit(train_X, train_y)
```

- alpha
 - default 는 1이며, 클수록 더 강한 정규화 적용



```
from sklearn.linear_model import ElasticNet

ElasticNet_reg = ElasticNet(alpha = 0.001, l1_ratio = 0.001)

ElasticNet_reg.fit(train_X, train_y)
```

- $l1_ratio$ = L1 정규화를 반영할 비율
 - 0에서 1사이 값을 가짐

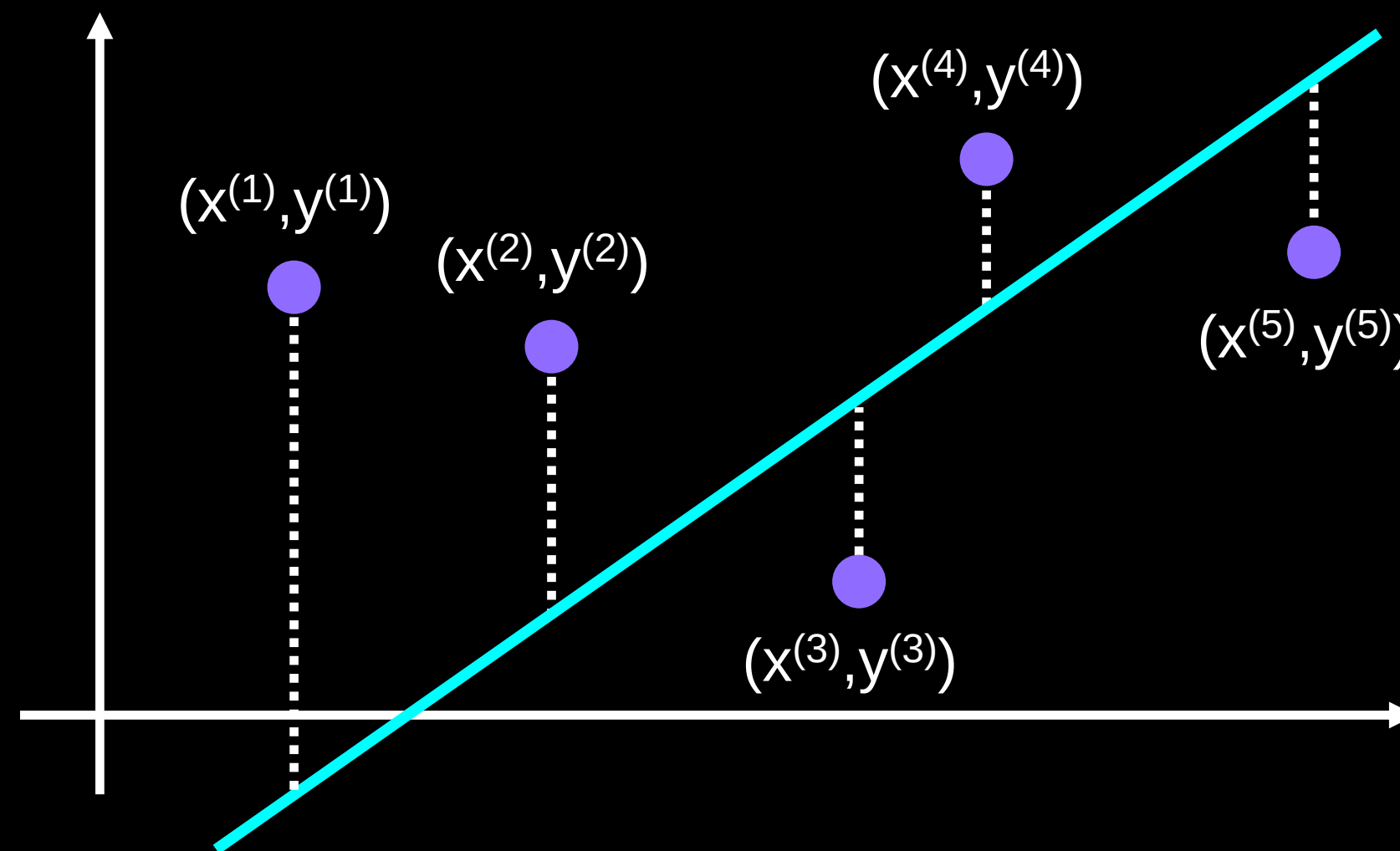
6. 회귀 알고리즘 평가 지표



- 어떤 모델이 좋은 모델인지를 어떻게 평가할 수 있을까?
 - 목표를 얼마나 잘 달성했는지 정도를 평가해야 함
- 회귀 분석 알고리즘의 목표
 - 실제 값과 모델이 예측하는 값의 차이가 크지 않도록 함



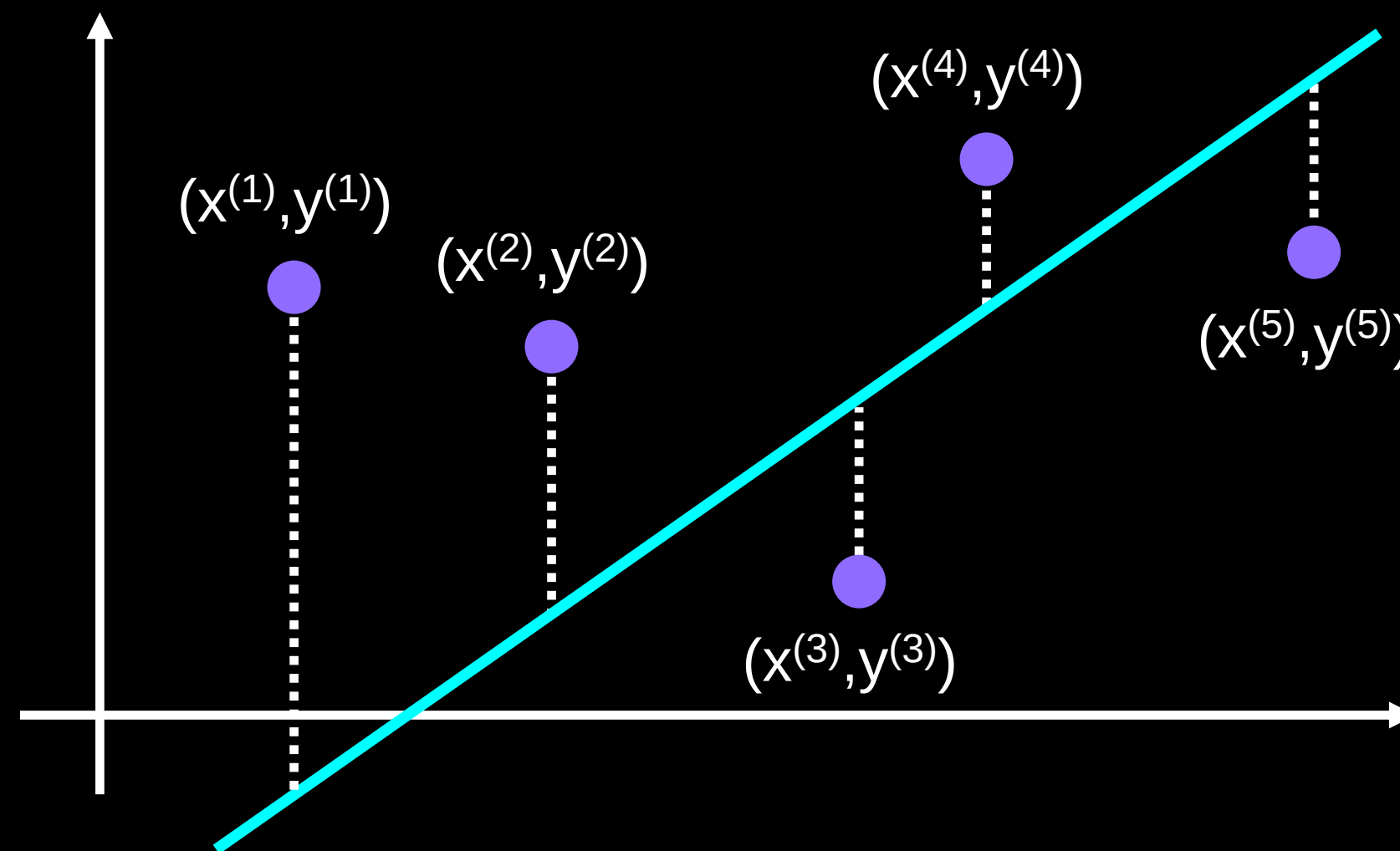
- 실제 값과 모델이 예측하는 값의 차이에 기반한 평가 방법 사용
- RSS, MSE, MAE, MAPE, R^2





- Residual Sum of Squares
- 실제 값과 예측 값의 단순 오차 제곱 합
- 값이 작을수록 모델의 성능이 높음

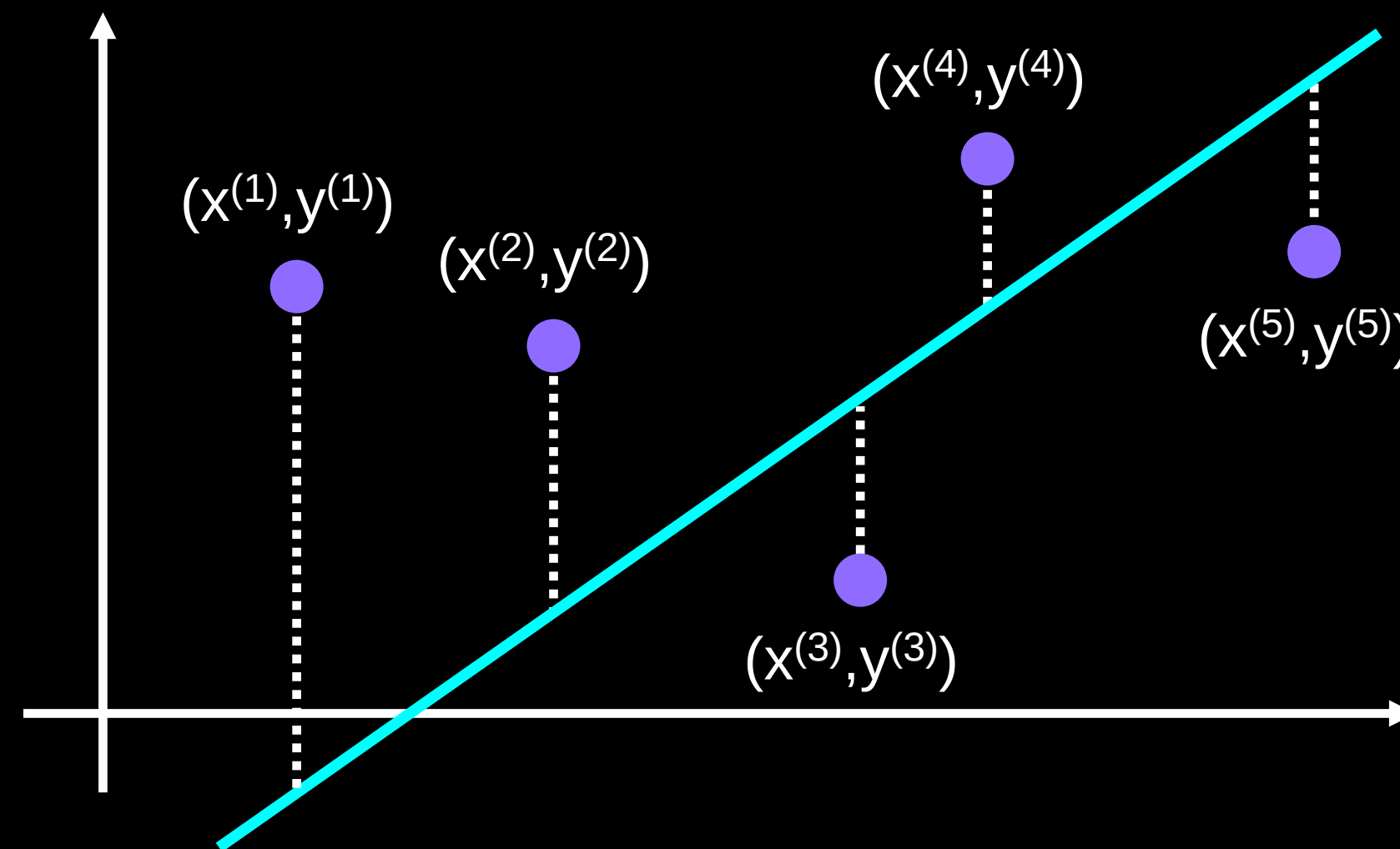
$$RSS = \sum (y_i - \hat{y}_i)^2$$





- 가장 간단한 평가 방법으로 직관적인 해석이 가능
- 오차를 그대로 이용하기 때문에 입력 값의 크기에 의존적

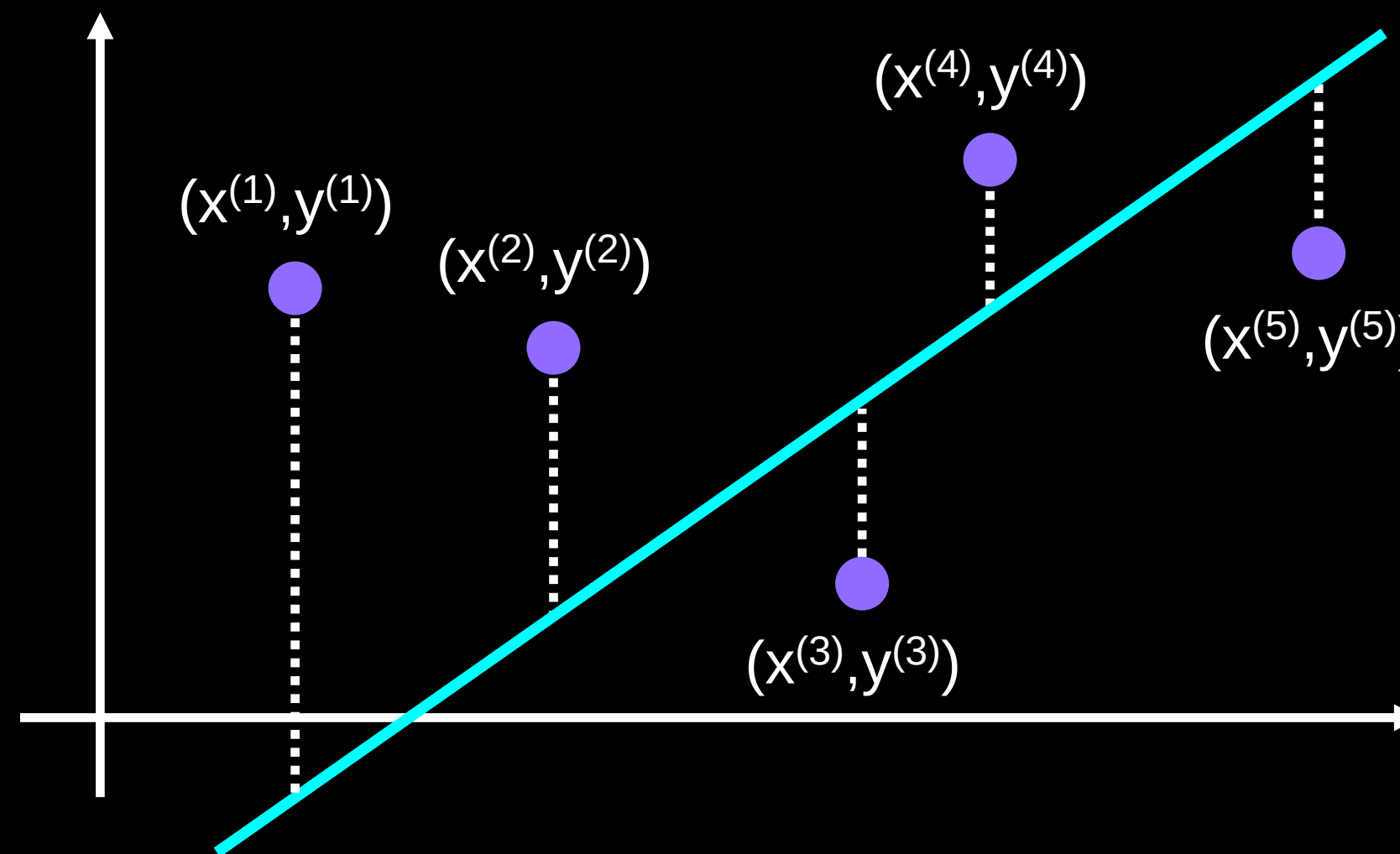
→ 이상치에 민감





- Mean Squared Error
- RSS에서 데이터 수만큼 나눈 값
- 값이 작을수록 모델의 성능이 높음

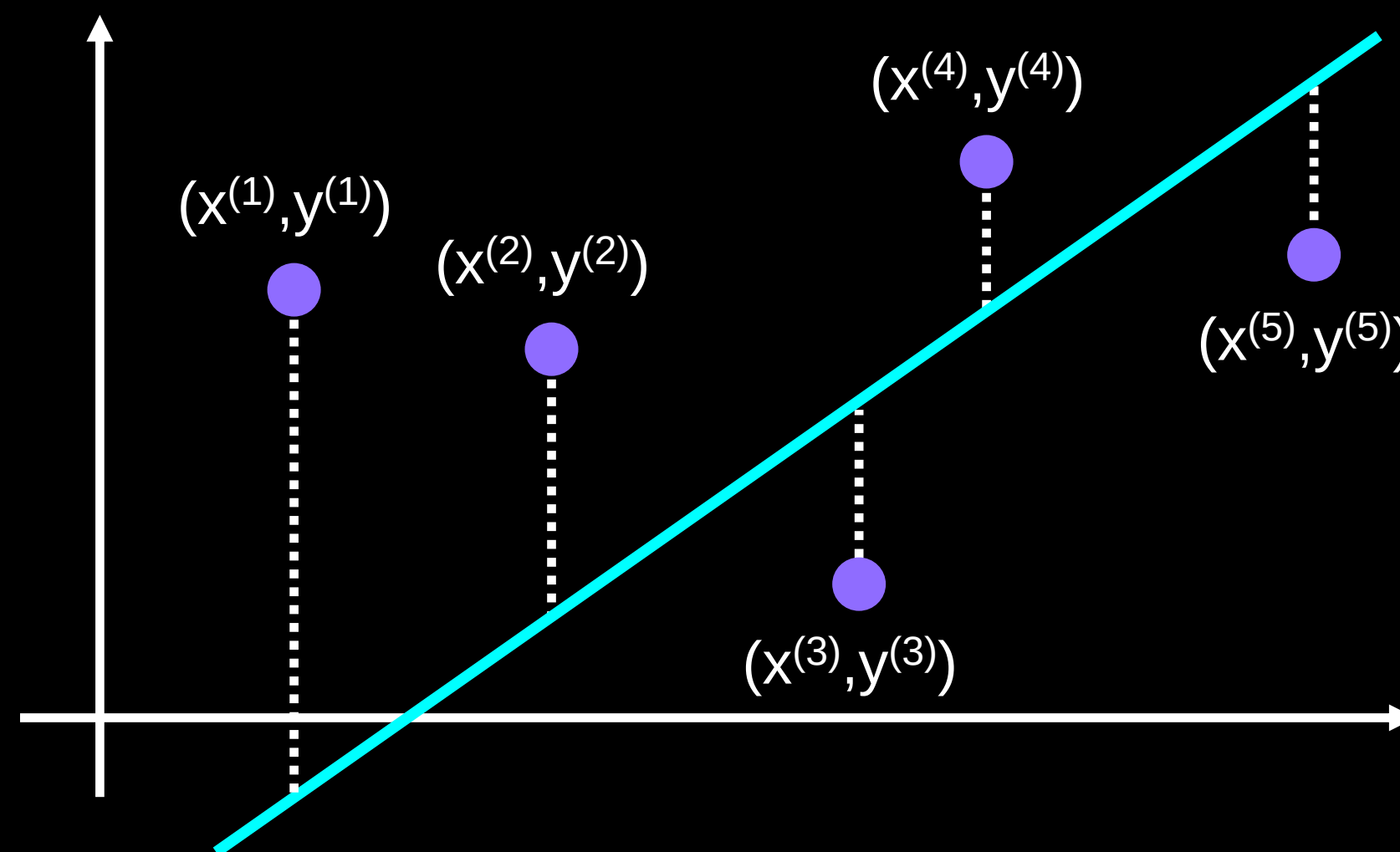
$$MSE = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$$





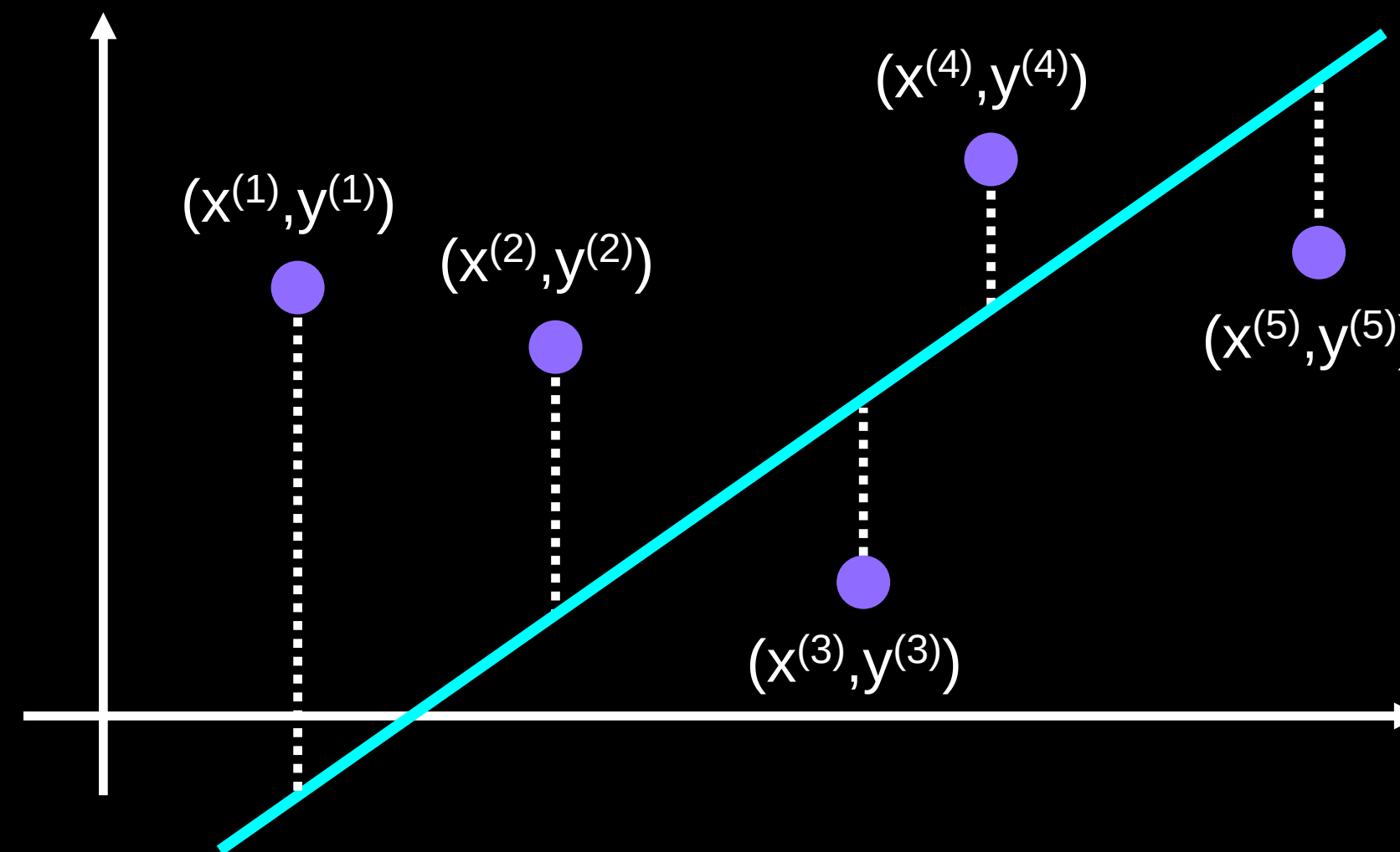
- Mean Absolute Error
- 실제 값과 예측 값의 오차 **절대값의 평균**
- **값이 작을수록 모델의 성능이 높음**

$$MAE = \frac{1}{N} \sum |y_i - \hat{y}_i|$$





- 간단한 평가 방법들로 직관적인 해석이 가능함
- MSE : 이상치에 민감
- MAE : 변동성이 큰 지표와 낮은 지표를 같이 예측할 시 유용, **이상치에 상대적으로 덜 민감**





- 모델이 종속 변수의 변동을 얼마나 잘 설명하는지 나타내는 지표
- **1에 가까울수록 높은 성능의 모델**
 - 총 변동 (SST, Total Sum of Squares): 관측 값과 평균 간 차이의 제곱합
 - 오차 제곱합 (SSE, Sum of Squared Errors): 예측 값과 관측 값 간 차이의 제곱합 (RSS와 같은 지표)

$$R^2 = 1 - \frac{SSE}{SST}$$



- $R^2 = 1$ 이면 모델이 데이터를 완벽하게 설명함을 의미한다.
- $R^2 = 0$ 이면 모델이 모든 입력에 대해 학습 데이터의 평균값을 예측한다는 의미이다.
- 아래 수식을 통해 도출한 R^2 는 음수가 될 수도 있으며, 이는 모델이 평균값을 예측하는 것 보다는 못한 예측을 한다는 의미이다.
- 백분율로 표현하기 때문에 데이터의 크기에 의존적이지 않다는 장점이 있지만, 총 변동이 0일 경우 계산할 수 없다는 단점이 있다.

$$R^2 = 1 - \frac{SSE}{SST}$$



```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
# 검증 데이터에 대한 예측
```

```
y_pred = model.predict(X_valid)
```

```
# MSE 계산
```

```
mse = mean_squared_error(y_valid, y_pred)
```

```
print("Mean Squared Error (MSE):", mse)
```

```
# MAE 계산
```

```
mae = mean_absolute_error(y_valid, y_pred)
```

```
print("Mean Absolute Error (MAE):", mae)
```

```
# R-squared 계산
```

```
r_squared = r2_score(y_valid, y_pred)
```

```
print("R-squared:", r_squared)
```

- **RSS의 경우 직접 계산하는 방식**으로 구현 가능
- MSE, MAE, R-Squared 는 **sklearn.metrics** 를 이용해 구할 수 있음
- 실제값과 예측값을 전달하여 확인할 수 있음